



LCI EDUCATION
NETWORK
MEMBER

.VERITAS
UNIVERSIDAD



Escuela de
Animación Digital

UNIVERSIDAD VERITAS

Escuela de Animación Digital

IZI RIG

Proyecto de graduación para optar al grado de Licenciatura en Animación Digital

Alberto Zúñiga Sánchez

201430005

Tutores

Estela Ugalde Ortiz

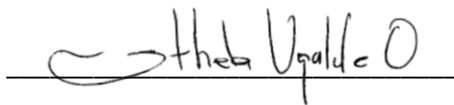
Ricardo Quirós Montero

II Cuatrimestre, 2021

Página de Aprobación

Como tutores del proyecto de graduación titulado IZI RIG, del estudiante Alberto Zúñiga Sánchez, requisito para optar por el Grado de Licenciado en Animación Digital, consideramos que el proyecto cumple con los estándares necesarios para presentarse a defensa pública, damos fe, que no existe presencia de plagio en el presente documento.

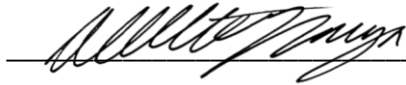
Tutores:

 Estela Ugalde Ortiz

 Ricardo Quirós Montero

Declaración Jurada

El abajo firmante Alberto Manuel Zúñiga Sánchez, cédula de identidad número 604240343 declara bajo juramento ser el autor del presente trabajo de graduación: Izi Rig para optar por el Grado de Licenciado en Animación Digital.



Alberto Manuel Zúñiga Sánchez

Cédula: 604240343

Agradecimientos

A todas las personas que aportaron un granito de arena para que este proyecto pudiera ver la luz.

Tabla de Contenido

CAPITULO I.....	7
Introducción	7
El Rig Como Soporte Invisible.....	7
Concepto inicial	8
Justificación	9
Tema.....	10
Objetivo general	11
Objetivos específicos	11
Delimitaciones	11
CAPITULO II.....	12
Marco contextual	12
Código, Rig e Interfaz	12
Frontend y Backend	13
Modular	13
Markers.....	14
Versión Alfa.....	14
Referencias Estéticas	15
Auto-rig Pro	15
Mixamo	17
Rapid Rig Modular	18
El Minimalismo de Rigomator	19
Referencias de carácter técnico.....	20
Bjørn Blaabjerg Sørensen.....	20
Xiong Lin	22
CAPITULO III.....	23
Proceso de Desarrollo de la Interfaz	23
Determinación de requisitos para la interfaz gráfica	24

Diseño de la Interfaz Gráfica	28
Implementación de la Interfaz Gráfica	33
Proceso de Desarrollo del <i>Rig</i>	37
Requisitos Del Rig	37
Diseño del Rig	42
Implementación del Rig	44
Localización de los Joints en la Posición Correcta	44
Creación del Esqueleto, Controles y Atributos	45
Documentación	48
Manual de Instalación	48
manual de Usuario	49
CAPITULO IV	56
Conclusiones y Recomendaciones	56
Problemas Durante la Implementación de la Interfaz	56
Manipulación de los Markers	57
Conclusiones la Interfaz	58
Conclusiones del Rig	59
Conclusiones	62
CAPITULO V	64
Bibliografía	64
CAPITULO VI	65
Anexos	65

CAPITULO I

Introducción

El Rig Como Soporte Invisible

Desde antes de ingresar a la carrera había descubierto la técnica de animación *stop-motion*, me interesé mucho por el sistema que lograba hacer tan fluidos los movimientos de aquellos inanimados muñecos. Cuando por primera vez pude utilizar un *rig*, más que asombrarme por lograr darle movimiento al personaje; me intrigaba como era posible que esto funcionara, poco a poco este pensamiento se convirtió en un fuerte deseo por crear sistemas que lograran manipular todo tipo de personajes, tales como: humanos, animales, mecanismos complejos y gigantescos dragones.

Se podría decir que el *rig* es el alma de la animación en sí, digital o física; así como toda marioneta tiene cruceta e hilos, en la animación todo objeto o personaje tiene detrás un mecanismo complejo que le permite cobrar vida. La persona encargada de otorgar esta vitalidad, el animador, está altamente familiarizado con este sistema, es su herramienta de trabajo y sin él le sería imposible crear movimiento; en cambio, el espectador solo logra ver el producto final, dejando al *rig* como algo imaginativo y a veces inexistente, sin embargo, extremadamente necesario.

El animador promedio no se dedica al *rigging* como tal, este prefiere en su mayoría el uso de herramientas que le faciliten la creación del *rig*, dichas herramientas se denominan *auto-rigs* y distan mucho de su nombre, ya que no son mágicas ni automáticas, aunque si logran asistir al usuario para que la experiencia en la elaboración del *rig* sea más sencilla, facilitando y simplificando la tarea.

El ser humano no solo tiene la capacidad de comunicarse entre nuestra misma especie, sino que también con otras especies, esta extensión no limita la comunicación entre los diferentes seres vivos, sino

que también se ha desarrollado la competencia de comunicarnos con las máquinas, y como se puede justificar esta acción, para esto se puede explicar mediante el siguiente ejemplo:

Cuando se utiliza una maquinaria mecánica el operador manipula de manera directa los mecanismos necesarios para crear una acción, hay un vínculo directo entre el usuario y el mecanismo de acción. En la mayoría de los sistemas digitales no existe un mecanismo físico que le permita al usuario manipular una acción directamente. Imaginemos la acción física de colgar un cuadro en una pared versus la acción digital de publicar una foto en internet, en el primer caso todo puede hacerse por la persona en cuestión, pero en el segundo caso no, el usuario que desea subir una foto debe comunicarse con el sistema para que este se encargue de publicar la fotografía, dicha facilidad informática es gracias a la interfaz de usuario.

Una computadora es capaz de procesar datos, entenderlos y con ellos crear una respuesta lógica esperada. El lenguaje de programación es la manera que emplea un sistema para comunicarle de manera lógica una acción al procesador. Es gracias a la programación que el usuario puede interactuar con un programa. Detrás de toda interfaz de usuario y de toda acción que realice un sistema hay código.

Así como no es posible explicar una receta de cocina sin mencionar sus ingredientes, nos es unimaginable exponer un *auto-rig* sin nombrar; el *rig*, la interfaz y el código.

Concepto inicial

El proceso de *rig* es complejo, y debe ser preciso, un pequeño error puede hacer que todo el sistema colapse y no funcione. Es común que los pasos para realizar un sistema se repitan varias veces, por lo que el *rigger* puede utilizar diferentes formas de realizar estos pasos, entre ellas destaca el código, utilizado para crear sistemas efectivos, más rápidos y simplificados.

Luego de un tiempo realizando diferentes sistemas de *rig*, se logró almacenar una cantidad grande de códigos útiles para diferentes procesos de *rigging*, por lo que surgió la idea de unificarlos y facilitar el proceso de *rigging* manual.

Con lo expuesto anteriormente surge la idea de crear una interfaz para dichos códigos, así me aventure en el mundo de la programación y de la creación de herramientas, en este caso, usando el lenguaje de programación y el programa de animación MAYA.

Después de investigar las diferentes alternativas de *auto-rigs* que existen, se visualiza que en su gran mayoría son toscas, tienen fallos y no son agradables con el usuario, de igual manera están desactualizados, usan métodos antiguos y aunque el usuario que las usa no logra darse cuenta de esto, le ocasiona problemas a posterior que no logran solucionar por falta de conocimiento, es de aquí que nace no solo la idea de crear una interfaz para los códigos de *auto-rig* sino también la creación de una herramienta que use métodos sencillos y fáciles de comprender por el usuario.

Se pretende:

- Elaborar una herramienta con una interfaz minimalista que no abrume con tecnicismos al usuario.
- Que el uso de la herramienta requiere de una cantidad pequeña de pasos cortos y sencillos.
- Que la herramienta logre general un mecanismo estándar y funcional.

Justificación

La principal motivación del proyecto es dar paso a una carrera cien por ciento profesional, por esto mismo se optó por un proyecto cuya temática refleje el sector de la animación digital en el que el estudiante se ha propuesto masterizar. Dicho objetivo abre la oportunidad para generar un portfolio técnico y practico eficaz al establecer las bases a la salida en el mercado comercial.

Una vez que se define la temática en la que se fundamenta el proyecto, se investiga y analizan las problemáticas sin resolver que existen en la esfera del CGI. Como resultado de un análisis empírico y una fuerte motivación personal se escoge el desarrollo de un plugin para maya que cree de manera automática un rig para el animador. Más adelante (marzo 2020) se investiga parte del mercado y la competencia, se concluyó que el mercado no tiene un plugin amigable con el usuario y que es su mayoría estos están hechos para personas con un conocimiento intermedio y avanzado de rig.

Izi Rig pretende tomar las debilidades de la competencia y convertirlo en sus fortalezas. Tenemos el caso de Mixamo, nuestro máximo exponente en lo que facilidad y estética se refiere. Sencillo, rápido y funcional, pero debido a sus escasos ajustes genera muchos errores; solo es útil para animaciones muy básicas, personajes simples y escenas con poco detalle e importancia. Izi Rig no solo aspira a ser sencillo, rápido y funcional; gracias a que es un plugin dentro de maya, cuenta con más estabilidad, solucionando esta carencia de Mixamo.

Es debido a que actualidad las herramientas que generan un rig de manera automática no son versátiles, hacen que este mismo sea algo tosco y / o difícil de maniobrar al momento de su utilización, también cabe destacar la escasa competencia estética que existe en la actualidad, genera la necesidad de diseñar una herramienta de *rig* automática que solucione:

- La carencia de una interfaz intuitiva.
- Un producto usable solamente como relleno y no para tomas complejas.
- Pocos ajustes a la hora de la creación del *rig*.
- Software defectuoso o desactualizado.

Tema

Desarrollo de una herramienta hasta su versión alfa que genere un *rig* de manera automática empleando lenguaje de programación, para Autodesk Maya 2020.

Objetivo general

Desarrollar una herramienta para el programa Autodesk Maya 2020, sin incluir su etapa de pruebas.

Objetivos específicos

1. Implementar una herramienta que cuente con una interfaz amigable con el usuario, sin incluir su etapa de pruebas.
2. Ejemplificar el uso de la herramienta y su desempeño en diferentes tipos de personajes.
3. Desarrollar una imagen de la herramienta que sirva como portfolio para la búsqueda de nuevos proyectos.

Delimitaciones

1. La elección de un proyecto cuyas bases son ajenas a la carrera de animación digital, esta delimitación se vio presente en distintas áreas.
 - 1.1. Carencia en los conocimientos de UX/UI, por lo que se tuvo que consultar a profesionales del área para lograr un resultado adecuado a la calidad del proyecto.
 - 1.2. Falta de conocimiento de programación para resolver problemas relativamente sencillos.
 - 1.3. Falta de experiencia con problemas matemáticos avanzados en informática.
2. Constantes desactualizaciones del software utilizado.
 - 2.1. Maya cambio para el 2021 la versión de Python utilizada.
 - 2.2. Ocurrieron cambios grandes en la manera de elaborar un *rig* con Maya 2022.

CAPITULO II

Para tener una idea más clara sobre lo que se expone en el documento se comentan conceptos generales importantes para la comprensión del documento que se usan recurrentemente.

Siempre es de gran ayuda analizar los referentes que se mantienen en la elaboración del proyecto, en este caso tenemos 2 grades grupos a observar, por un lado, en el aspecto visual de la herramienta y por otro lado el aspecto técnico.

Marco contextual

Código, Rig e Interfaz

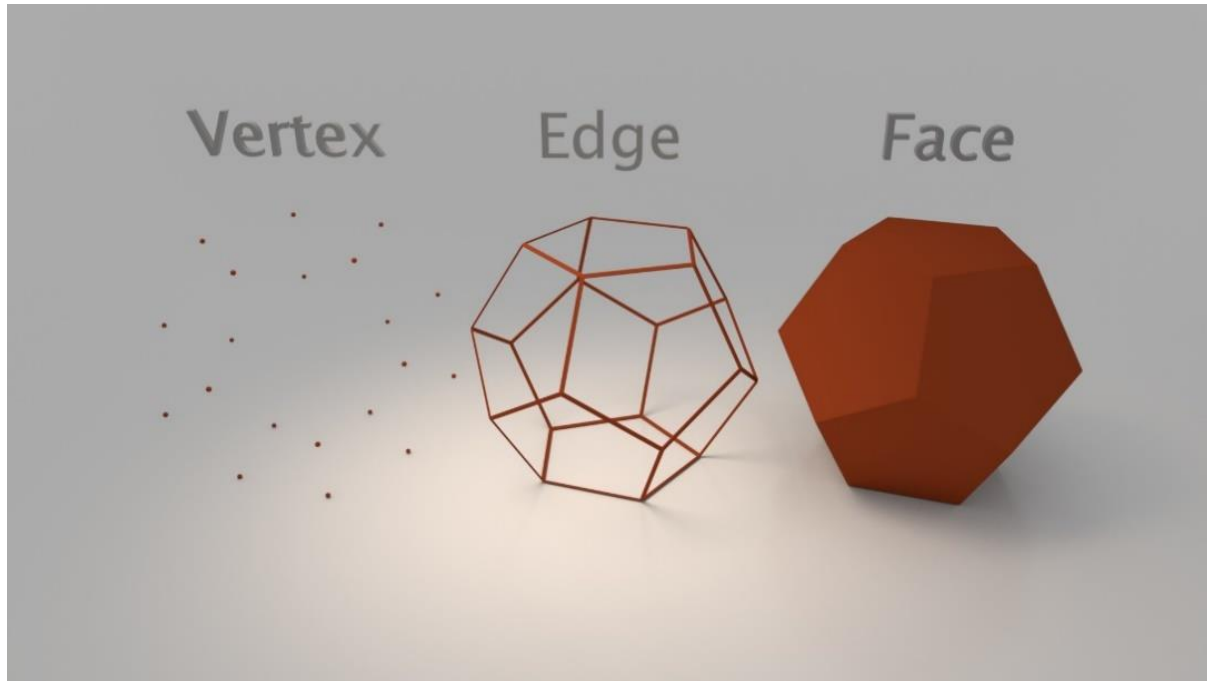
Estos 3 conceptos son la base del proyecto: el *rig* es por obviedad la meta a la que se pretende llegar; sin el código seria completamente imposible crear un software y la interfaz es aquella que nos permite poder utilizar la herramienta.

Para entender el impacto que tiene el *rigging* en la industria de la animación ocupamos comprender como funciona un espacio de tres dimensiones para una computadora. Un ordenador es incapaz de ver y razonar como lo hace un humano, para lo que nosotros son objetos en un espacio “vacío”, es para la computadora equivalente a un mapeo de coordenadas sobre tres ejes. Esto lo hacen los computadores para lograr posicionarse en un punto determinado dentro de un área infinita, estos ejes comúnmente son: X, Y y Z.

Una geometría como la conocemos dentro de los programas de animación y modelado 3D, no es más que una compilación de puntos en el espacio. Ejemplo: primero se le proporciona al programa 20 coordenadas distintas, cada una de estas nos crea un punto en un espacio establecido. Se le ordena al sistema que trace una línea entre los puntos adyacentes, dando como resultado 30 trazos conectores, los cuales a su vez conforman una malla. Luego al cubrir el área resultante entre los trazos de la estructura, se obtiene un sólido volumétrico.

De esta manera se logra crear un Dodecaedro en un entorno virtual.

Figura 1



Render de la ejemplificación: vertex, Edge, fase

Frontend y Backend

La interfaz y el código que manipula el usuario es un espacio separado al código que internamente se corre en el programa. Este ambiente manipulable por el usuario engloba el Frontend, el resto del código y arquitectura del programa hace referencia al backend.

Modular

La mayoría de las herramientas para rigeo automático que encontramos en el mercado se limitan a la creación de un esqueleto predeterminado y estandarizado, no se permita la generación de extremidades extras o personalizado.

Si se le da al usuario la opción de crear, eliminar o editar de manera individual cada extremidad, podemos definir cada parte del todo como un módulo. Cabe recalcar que un módulo debe tener la posibilidad de usarse de manera independiente y ajena al conjunto.

Izi Rig Pretende tener una serie de módulos capaces de generar cada tipo de extremidad que el usuario necesite y la posibilidad de personalizarla a la medida. Esta funcionalidad está planificada para ser implementada poco a poco con cada actualización, por lo que se estipula será a partir de la versión dos.

Los módulos planificados se dividen en 3 categorías principales. Izi Rig simple, Izi Rig complex y Izi Rig archive. Cada módulo tiene a su vez una cantidad de submódulos para cada categoría.

Markers

Durante la utilización de Izi rig, el usuario debe señalarle a la herramienta puntos concretos que más adelante el software empleara para la creación del rig. El paso descrito anteriormente se puede realizar gracias a una marca estilo cruceta, la cual el usuario podrá mover libremente en el eje X y Y dentro de la interfaz de la herramienta. Ver Figura 2.

Cada cruceta se denomina *marker* y esta etiquetada con su correspondiente posición en el personaje a rigear.

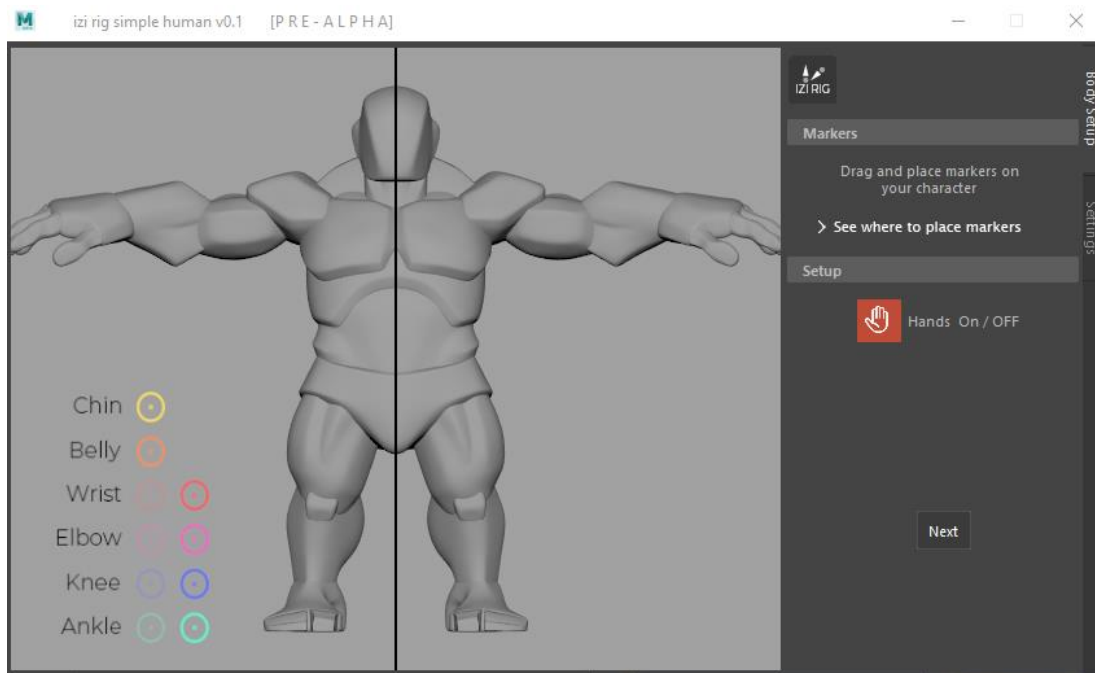
Izi rig trabaja con 6 *markers* principales para el cuerpo: *chin*, *belly button*, *wrist*, *elbow*, *knee* y *ankle*. Se pretende para la versión dos disponer de *markers* extra: *shoulder*, *hip* y *clavicle*. Para el posicionamiento de la mano también se cuenta con una serie de *markers* para la colocación correcta de cada Falange, estos serían 2 para dedo de la mano, uno en la base de este y otro en la articulación interfalángica distal.

Versión Alfa

El proceso de desarrollo de un programa tiene varias etapas, en el caso del Izi Rig como proyecto final de graduación se menciona como una versión alfa, aunque el proyecto Izi Rig tenga un alcance planificado hasta su culminación.

Cuando un programa en desarrollo está completamente listo para su utilización no es lanzado directamente al mercado, primero pasa por una serie de pruebas, la primera versión de este programa suele llamarse versión alfa.

Figura 2



Ejemplificación de los markers en la interfaz de Izi Rig simple human.

Referencias Estéticas

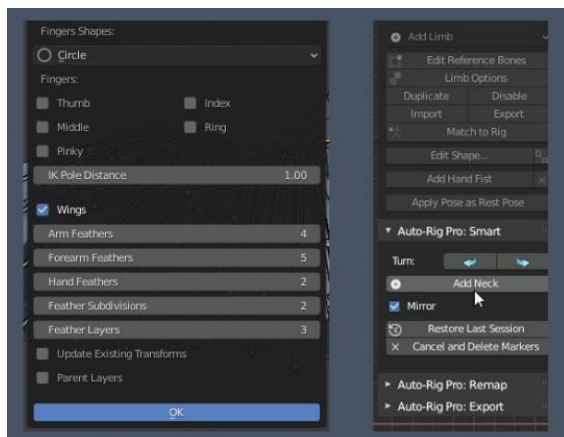
Auto-rig Pro

Plugin para la realización de un *rig* únicamente utilizando el *software* blender, esta herramienta permite al usuario crear diferentes tipos de mecanismos, tanto bípedos como modulares. Es desarrollado por Lucas Veber, animador, iluminador, modelador y *rigger* de personajes.

Interfaz de Usuario de Auto-rig Pro. Para la realización de un *rig* bípedo humanoide, el Auto-rig Pro cuenta con una función de reconocimiento inteligente, el plugin necesita que el usuario delimite ciertas áreas del cuerpo del modelo, el usuario acepta en la interfaz el paso siguiente y la herramienta automáticamente crea el sistema de huesos y articulaciones. Este sistema inteligente funciona únicamente en bípedos humanoides y su manera de utilización se da desde una vista totalmente frontal del personaje.

En la figura 3 podemos ver como la interfaz que nos proporciona Auto-rig Pro no es minimalista, sin embargo, esta implementa una paleta de colores limitada y desaturada, esta gama cromática es empleada a la hora de resaltar y delimitar áreas o botones de la interfaz.

Figura 3



Recorte de dos diferentes menús de Auto-rig Pro y su paleta de colores.

Nota. Adaptada de Rig fast with the Smart feature y Wings limb for advanced feathers rig [gif], por Artell, 2017, Blendermarket (<https://blendermarket.com/products/auto-rig-pro?ref=46>). Todos los derechos reservados [2021] por The Blender Market, LLC. Adaptado con permiso del autor.

Características Extras del Plugin. Al ser una herramienta modular cuenta con diferentes bloques de trabajo; caben resaltar los siguientes: un apartado dedicado a la creación de extremidades aladas; una modalidad que permite elaborar cuerdas, tentáculo y otras extremidades homólogas; y una modalidad para exportar el modelo a *software* especializado en la creación de videojuegos.

Aunque la herramienta es bastante completa, debido a la diferencia de *software* no será tomada en cuenta como referencia técnica ni práctica.

Mixamo

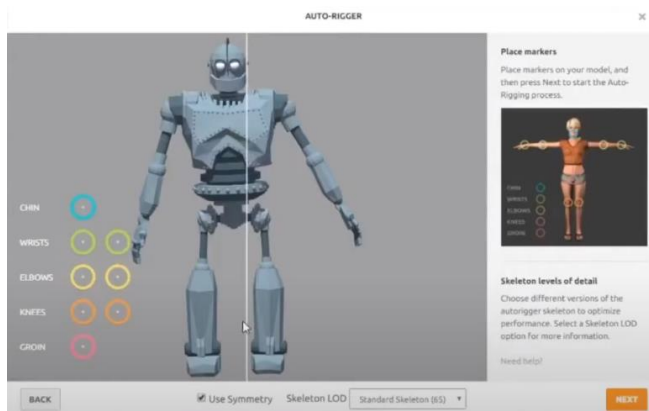
Es una herramienta que brinda un servicio en la web. El usuario requiere de conexión a internet para poder cargar el personaje a la plataforma, ahí mismo este sigue una serie de pasos para la configuración del *rig*, al finalizar el usuario puede descargar el personaje con el *rig* completamente listo y usarlo en Maya u otro programa.

Interfaz de Usuario de Mixamo y su Modo de Empleo. Mixamo cuenta con una interfaz muy limpia y sencilla de usar, esta guía al usuario paso a paso de manera ordenada, si no se ha terminado correctamente un objetivo previo la página no pasara al siguiente bloque. Al igual que el Auto-rig Pro, Mixamo cuenta con una serie de guías visuales que el usuario acomodara en las articulaciones correspondientes del personaje.

Por estas características es que Mixamo es el referente visual y estructural de mayor importancia para el proyecto. Izi Rig se enfoca en la practicidad y rapidez con la que el usuario puede posicionar las marcas visuales y tener un *rig* completamente funcional sin tener que salir de Maya.

Mixamo es desarrollado por la compañía Mixamo fundada en 2008 por Stefano Corazza y Nazim Kareemi. subsidiaria de la empresa Adobe.

Figura 4



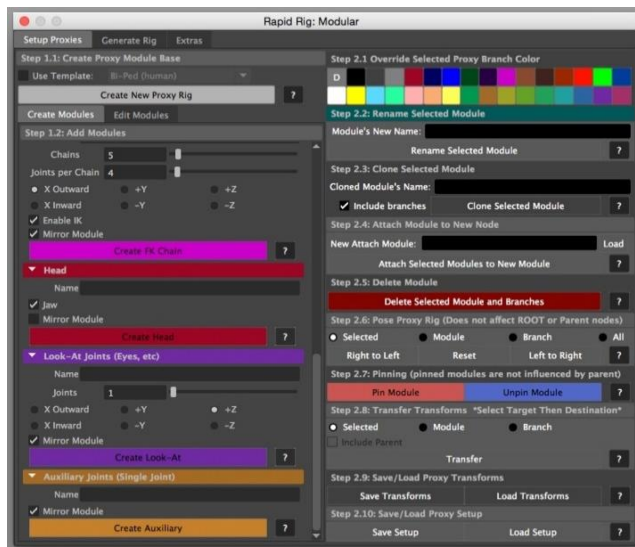
Representación visual de la interfaz de usuario de la página web Mixamo a la hora de posicionar los markers en el personaje.

Rapid Rig Modular

Esta herramienta de *rig* cuenta con tres series distintas, de las cuales dos siguen recibiendo actualizaciones periódicas y una, la más básica, ha sido abandonada por el creador y catalogada como obsoleta. Sin embargo, como referencia general se tomará en cuenta únicamente su versión modular. Este *auto-rig* se caracteriza por ser completo y muy efectivo, al ser modular permite la creación de cualquier criatura o personaje que se requiera trabajar. Rapid Rig es desarrollado por Dustin Nelson, él es un artista técnico y *rigger* en Electronic Arts.

Lamentablemente el Rapid Rig no es fácil de usar, su interfaz cuenta con una cantidad elevada de colores, términos y botones, por lo que abruma al usuario que no esté familiarizado con las terminologías utilizadas o que no tenga experiencia en la creación de *rigs*. Por lo tanto, su interfaz no será utilizada para referencia. Ver figura 5.

Figura 5

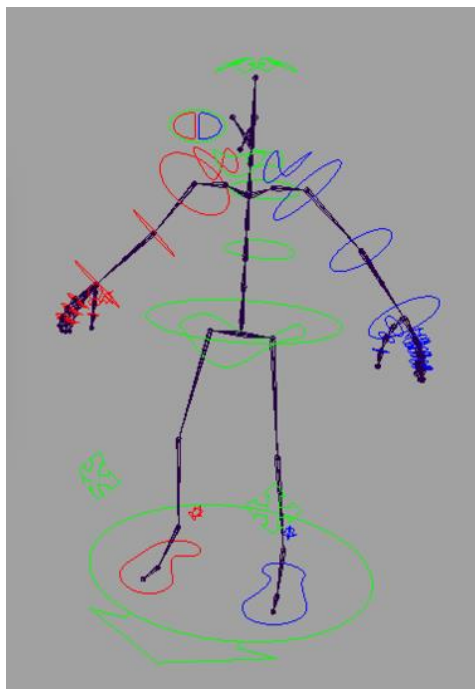


Representación visual de los menús que emplea la herramienta Rapid Rig.

Nota. Adaptada de Rapid Rig Modular: Modules Tutorial [video], por Dustin Nelson, 2016, Youtube (https://www.youtube.com/watch?v=sX_iUaE70iQ).

Dentro de la creación de Izi Rig la estética no está centrada meramente en la interfaz utilizable por el *rigger*, sino también al *rig* en sí; El *rig* es algo con lo que no es tan sencillo lidiar o mejorar, puesto que maya ya cuenta con una cierta disposición y dirección artística, no se puede modificar visualmente la mayoría de los elementos usados, pero se puede jugar con estos para crear una armonía. Es precisamente esto lo que se tomara en cuenta de *Rapid Rig*, como podemos ver en la figura 6, su *rig* es amigable de ver, no es recargado y sus controladores son fáciles de identificar.

Figura 6



Representación visual de los controladores que genera la herramienta Rapid Rig.

El Minimalismo de Rigomator

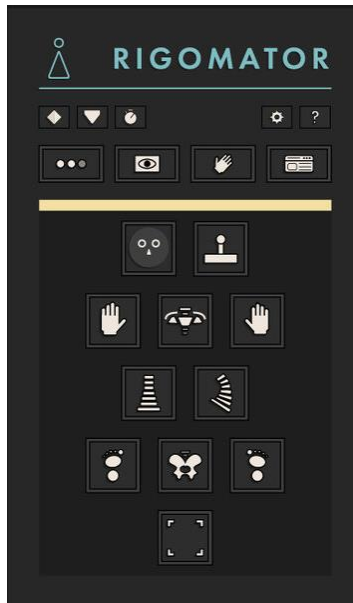
Es desarrollado para After Effects, es una herramienta que en comparación con las anteriores es sencilla y básica en sus funciones, sin embargo, su interfaz es minimalista y estéticamente compatible con la idea que se desea desarrollar con Izi Rig.

Rigomator es desarrollada por Bear Studios, una empresa independiente de animación y diseño, creada por Samuel Kim, diseñador gráfico especializado en *motion graphics*.

Características Extras del *Plugin*. Permite la exportación de animaciones y la creación de una biblioteca de animaciones para su posterior utilización, esto genera un ahorro en la cantidad de tiempo del animador.

Cabe resaltar que la herramienta se usa en un programa ajeno al tema del proyecto y que este genera un *rig* 2D, por lo que sus aspectos técnicos no son tan relevantes.

Figura 7



Referencia visual de la interfaz minimalista que es empleada por Rigomator.

Referencias de carácter técnico

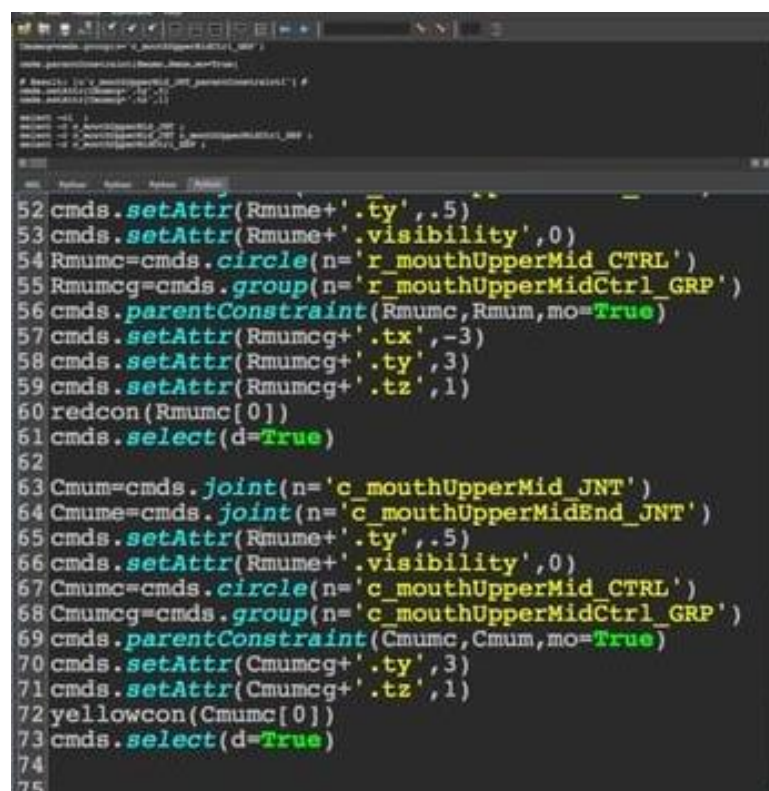
Bjørn Blaabjerg Sørensen

Nacido en 1988, Generalista CG oriundo de Dinamarca, se especializo en *Rigging* y *Scripting*. Licenciado en artes gráficas de The Animation Workshop. Trabaja actualmente en la industria VFX como

rigger y Generalista en Copenhague. Trabajó anteriormente para Double Negative, Storm Studios, MPC London e ILM London. Ilustrador 2D tradicional y se define como pintor competente.

Código de Python en Maya. Para el aspecto técnico de la creación de *Izi Rig* se tomó como referente la implementación que hace Bjørn Blaabjerg del lenguaje de programación Python en la consola de maya.

Figura 8

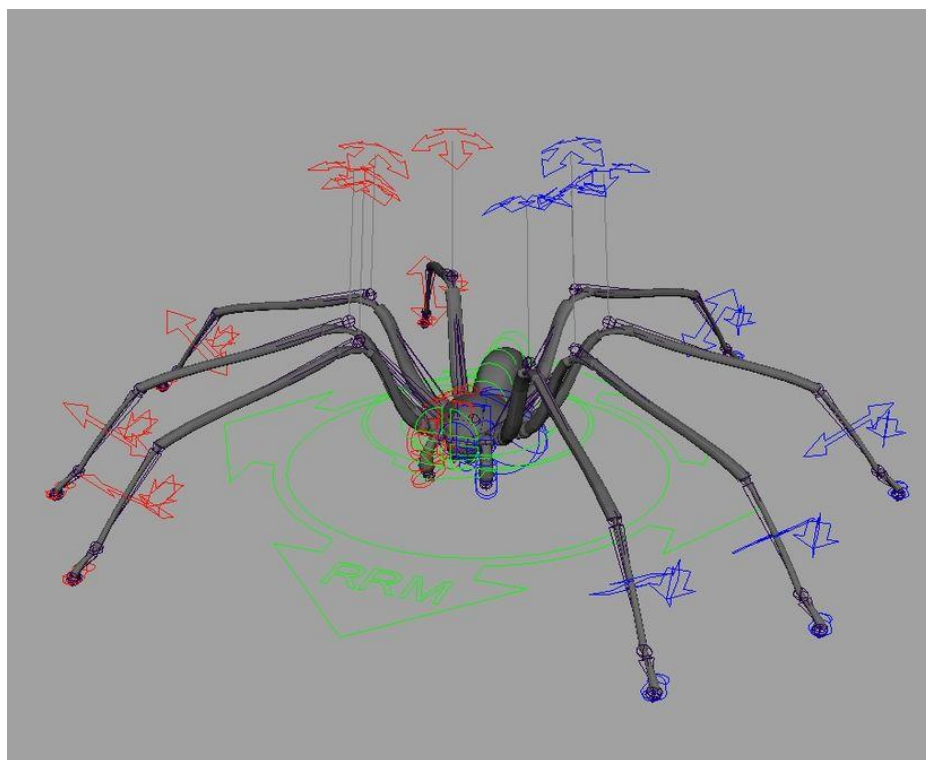


Referencia visual de la creación de un código Python dentro de la consola de maya.

La implementación modular de Rapid Rig. A la hora de la creación de un *rig*, la mayoría de *auto-rigs* crean un solo esquema dependiendo del personaje, en este caso un bípedo humanoide, queda en el usuario terminar de ajustar las articulaciones para crear el *rig*.

Como se indica en el inciso de estética, esta herramienta es mas de carácter técnico, por lo que se tomó como referencia el uso del modularidad para la creación de un *rig* más completo, complejo y personalizado. Ver figura 9.

Figura 9



Referencia visual de la creación de un rig modular con la herramienta Rapid Rig.

Xiong Lin

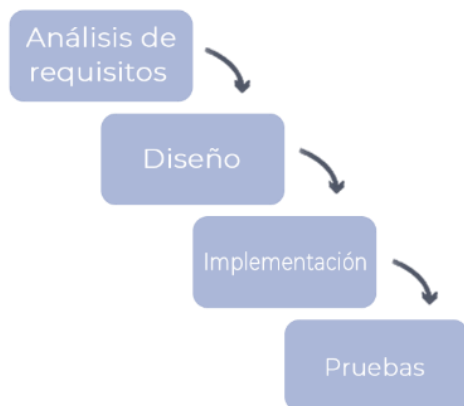
Especialista en *rigging set-up* y pintado de pesos, se describe a sí mismo como fluido en lenguajes de programación como: *Python, PySide/PyQt, Maya API and Maya widgets command for rigging pipeline development*. Modelador de personajes. Generalista CG con experiencia en animación CG. Actualmente trabaja en *BUCK Design LLC* como parte del personal de diseño de personajes.

CAPITULO III

El desarrollo de software como todo proceso de creación esta apegado a un método específico de trabajo, este consiste en la aplicación de prácticas estandarizadas para el desarrollo de aplicaciones. Como se puede ver en la figura 10, el método consiste en fases de desarrollo o de análisis, en el caso del modelo tipo cascada cada fase debe estar finalizada para poder dar pie a la fase siguiente.

Con el fin de desarrollar de mejor manera el proyecto, se separa en 3 campos: el ciclo de la interfaz, el proceso de *rig* y el proceso de implementación detallado.

Figura 10



Referencia visual del Systems development life cycle (SDLC) en su metodología tipo cascada hasta la fase alfa.

Proceso de Desarrollo de la Interfaz

Cuando se interactúa con un programa cada acción que el usuario realiza se da gracias a la interfaz, esta es la única ruta que se tiene para poder comunicarse de manera correcta con el software y es de vital importancia que en el desarrollo de este se planifique y se diseñe basándose en la experiencia del usuario.

Es por la idea anterior que Izi Rig está enfocado es su facilidad de uso y su buen diseño estético.

Determinación de requisitos para la interfaz gráfica

Todo *software* comienza como una idea, esta idea se debe moldear y planificar antes de poder realizarla, en el SDLC la primera fase es un análisis exhaustivo de la idea. Las cuales están son: *Content Map* y el *Lifecycle*.

Content Map. Se procede a listar uno a uno cada acción que se requiere el software alcance a realizar. Cuando se realiza el desglose se debe tener en cuenta únicamente el objetivo de la acción que este debe generar y se evita usar métodos de realización concretos.

A continuación, se visualiza la ejemplificación del uso de Izi Rig en el *Content Map*, Inicio de *rig*, los ajustes pertinentes y finalización:

Izi Rig Simple Content Map

Esta herramienta permite al usuario elaborar el *rig* de un personaje humanoide. Para la creación del *rig* simple cada parte solo funcionara si el contenido anterior es realizado, con excepción de los contenidos que sean reiterativos.

Inicio del *Rig*.

El usuario debe abrir la interfaz del plugin Izi Rig simple (este paso es realiza desde la interfaz de propia de MAYA, por lo que no se tomara en cuenta para el diseño).

1. Ajustes del personaje.

1.1. Se debe seleccionar el personaje al que se le hará el *rig*.

1.2. Se visualiza el personaje de forma frontal. No se puede rotar, escalar ni mover al personaje libremente. Únicamente se puede rotar al personaje 90 grados en el eje x, y, z. Esta opción tiene como objetivo corregir la posición del personaje.

1.2.1. Se muestra al usuario la manera en la que se debe colocar el personaje para la realización del rig y se le dará una advertencia de posibles errores si el personaje no está en la posición correcta. Este contenido tiene la opción de no volver a mostrarse de manera automática. Durante los ajustes del personaje, el usuario puede acceder a este contenido en todo momento.

2. Ajustes del rig corporal.

2.1. Se elige si el *rig* se genera de manera simétrica o asimétrica. El usuario puede alternar entre las dos opciones constantemente.

2.2. Se generan de manera automática diez indicadores: una barbilla, un ombligo, dos muñecas, dos codos, dos rodillas y dos tobillos.

2.2.1. Se generan en un plano superpuesto a la vista frontal del personaje.

2.2.2. Pueden ser arrastrados por el usuario.

2.2.3. El usuario debe posicionar los indicadores sobre el personaje.

2.2.4. Cada indicador cuenta con una etiqueta para que el usuario sepa donde debe ser colocado.

2.2.5. El usuario tendrá la capacidad de duplicar cualquier indicador. Todo indicador puede ser duplicado un número infinito de veces.

2.2.6. Cada indicador puede ser eliminado por el usuario. Cuando un indicador de los 11 originales es eliminado, vuelve a su posición inicial.

2.2.7. Se elige si el personaje tiene manos.

3. La posición de los indicadores puede ser guardada para su uso posterior. También existe la opción de cargar.

- 3.1. Esto es realizado desde la ventana de búsqueda de maya.

4. Ajustes del *rig* manos.

- 4.1. Si el *rig* es asimétrico este contenido debe ser individual para cada lado.

- 4.2. Se debe elegir la cantidad de dedos que el personaje posee por cada mano.

- 4.2.1. Una mano puede tener de uno a cuatro dedos.

- 4.2.2. La mano puede no tener pulgar.

- 4.3. Se generan de manera automática veinte indicadores.

- 4.3.1. Se generan dos indicadores por cada dedo: pulgar, índice, medio, anular y meñique.

- 4.3.2. Se generan en un plano superpuesto a la vista superior de la mano.

- 4.3.3. Pueden ser arrastrados por el usuario.

- 4.3.4. El usuario debe posicionar los indicadores sobre el personaje.

- 4.3.5. Cada indicador cuenta con una etiqueta para que el usuario sepa donde debe ser colocado.

Finalización del *Rig*.

El usuario debe indicar que el *rig* está listo para su finalización.

Se le anuncia al usuario si el proceso realizado ha sido exitoso y que el *rig* está completo.

En caso de que falle el proceso, esto se le indica al usuario.

Lifecycle. Se realiza una lista de posibles usuarios que van a usar el programa, esta lista tiene como objetivo enumerar las necesidades de cada persona y así poder generar las posibles soluciones que se les puede ofrecer.

Etapa del <i>Lifecycle</i>		
Conciencia	Consideración	Decisión
Realiza y expresa síntomas de un potencial problema u objetivo posibles	Se define claramente la manera en la que se puede obtener la solución a los problemas u objetivos	Se define su estrategia de solución, método o enfoque
• Utilización de comandos sencillos o pocos pasos para realizar el <i>rig</i>. • <i>Rigs</i> automáticos suelen ser poco ajustables. • Las herramientas de <i>rig</i> automático no suelen tener <i>rig</i> facial. • Creación de diferentes <i>setups</i>, para elegir entre bípedo, cuadrúpedo, monstruo o creatura. • Añadir dinámicas.	• Crear una buena relación entre el usuario y la herramienta. • Desarrollar una interfaz intuitiva. • Personalización tanto de extremidades, apéndices y dinámicas para la creación de diferentes creaturas. • Implementar un <i>rig</i> facial automático con la posibilidad de personalizarlo	• Implementar pruebas de usabilidad. • Otorgar al núcleo de opciones base la posibilidad de edición con temas avanzados de manera intuitiva. • Incluir como parte del núcleo de opciones base la creación de un <i>rig</i> facial

El *Content Map* se crea de la manera que el diseñador considere más útil para su proceso de diseño, se puede basar por persona y tener uno distinto para cada una y eventualmente unir similitudes para implementar estrategias, métodos o soluciones. En este se puede trabajar con el *Lifecycle Stage* en el que se estudia los problemas u oportunidades que se presentan, luego se consideran, aclaran y definen los problemas u oportunidades y por último se toma la decisión de cuales soluciones implementar.

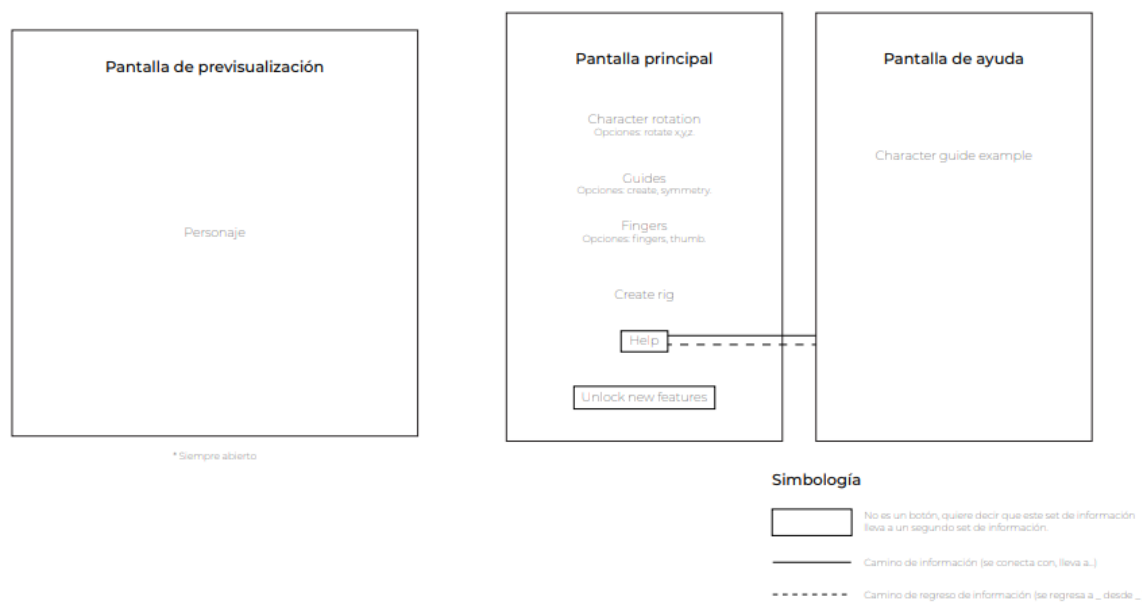
Diseño de la Interfaz Gráfica

Con los requerimientos del programa listados y los contenidos orientados de manera correcta se comienza la etapa del diseño.

Esta segunda etapa del proceso se centra en la búsqueda de soluciones graficas a los requerimientos del sistema.

Site Map. Ayuda al diseñador a ordenar de manera jerárquica y diamagrado las páginas, vínculos y etiquetas. Se puede visualizar de manera correcta el flujo de acciones que el usuario pueda hacer a la hora de usar el software.

Figura 11



Referencia visual del site map de Izi rig.

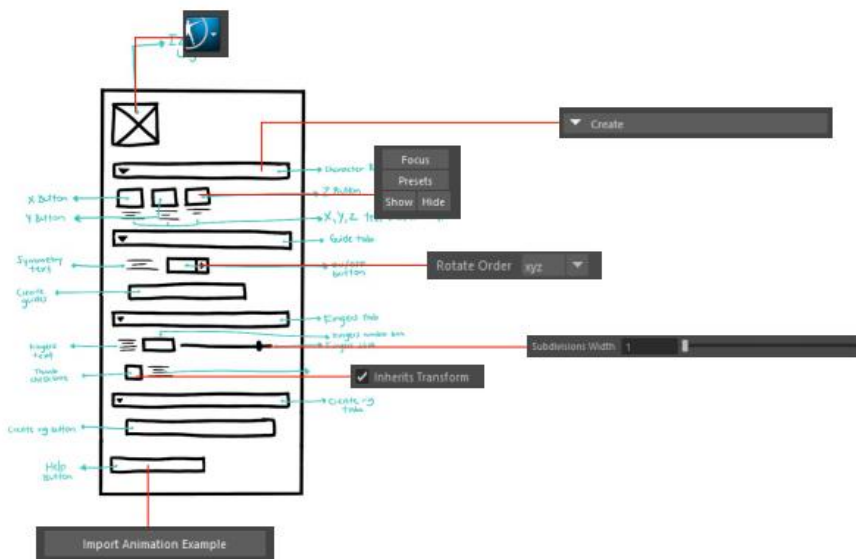
Nota. En anexos se encuentra una versión escalada para su mejor visualización.

Wireframe. Son posibles diseños de *layout* que apliquen para las personas o público meta, aún no se diseña con colores ni tipografías, pero se desarrollan las jerarquías propuestas anteriormente. Estos

se pueden cambiar cuantas veces sean necesarias y a partir de este punto se puede comenzar a probar el diseño con prototipos *lo-fi*.

Es importante que quede claro que este funciona como boceto y no como algo que el usuario final debe entender, ya que no es parte de este proceso del todo. El momento en el que el usuario entiendo cómo hacer el *rig* es en etapas de más alta fidelidad donde el *wireframe* se convierte en prácticamente el producto final.

Figura 12

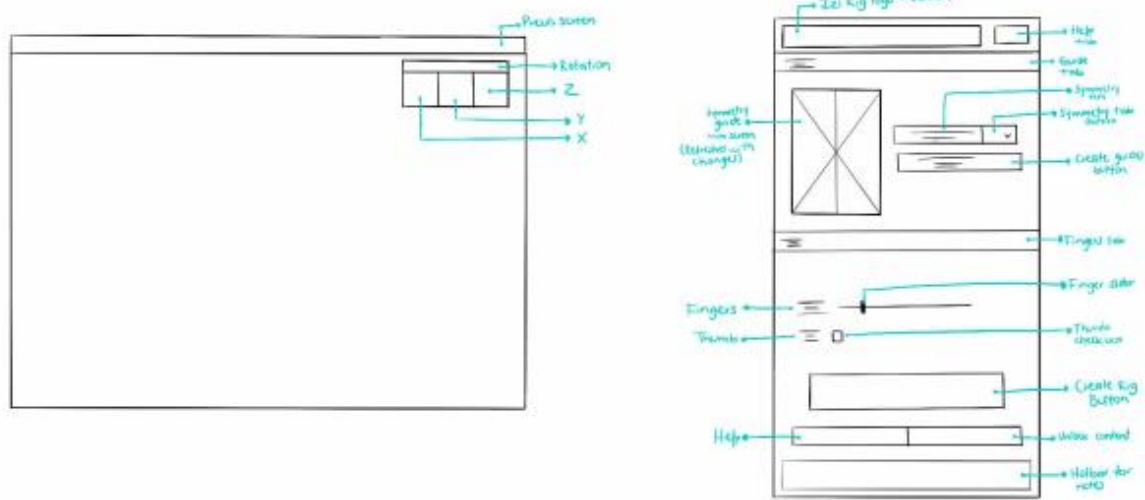


Referencia visual de un wireframe.

Nota. En anexos se encuentra una versión escalada para su mejor visualización.

Se realiza una exploración para la versión final de Izi Rig. Donde se utilizará un lenguaje de programación más amigable para crear interfaces personalizadas, lo cual hace que el tiempo de programación se extienda muchísimo más por lo que no es parte de la meta final de esta versión alfa.

Figura 13



Referencia visual del wireframe de Izi Rig.

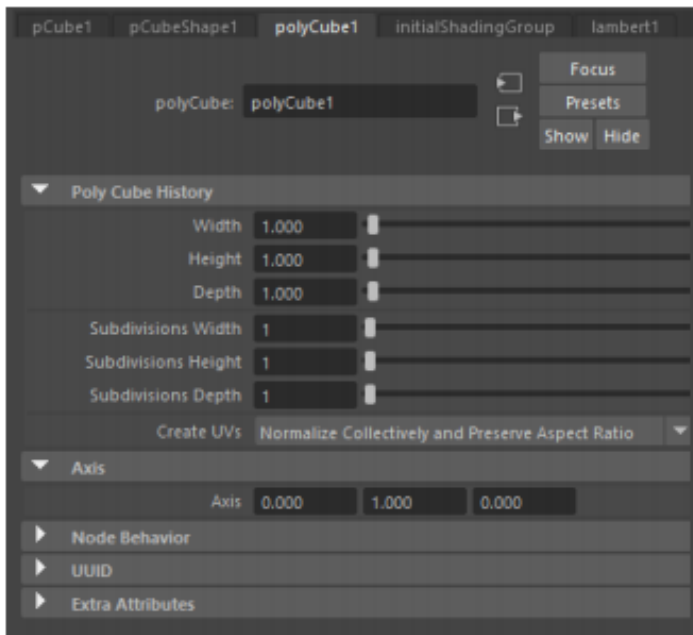
Nota. En anexos se encuentra una versión escalada para su mejor visualización.

Toolkit. Al ser programado directamente en Python Maya se restringe severamente lo que se puede crear gráficamente con el UI.

Dado este hecho el resultado grafico final no puede variar de lo que ya existe previamente en Maya, por ejemplo, los botones no pueden ser resaltados con un color distinto, la tipografía no puede cambiar a menos de que por alguna razón pueda ser cambiado por medio de código externo modificando Maya propiamente.

Por esto el UI a manera gráfica parece que está desarrollado por el equipo de Autodesk ya que en este momento del desarrollo no se va a trabajar con otros lenguajes de programación que permitan personalizar a medida otros elementos como slider o tipográficas nuevas.

Figura 14



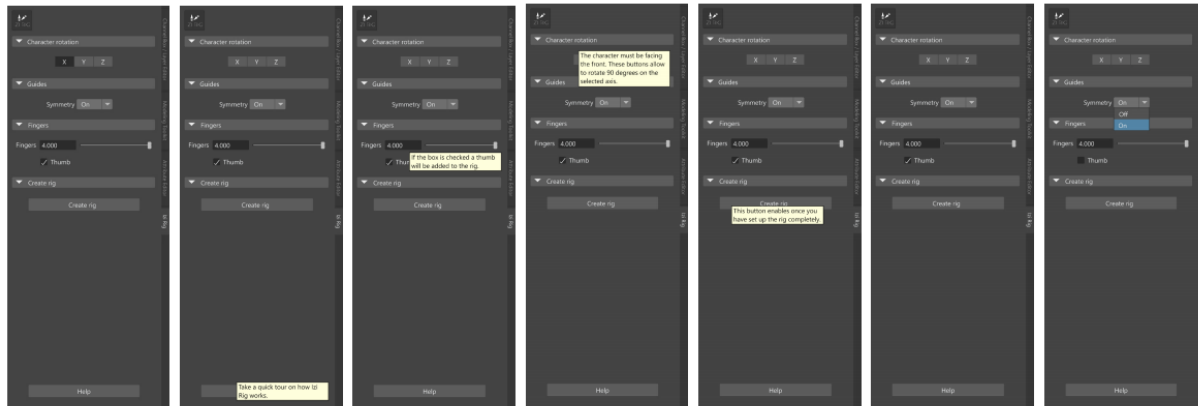
Referencia visual del toolkit que utiliza Autodesk Maya 2020.

No existen *sketches*, *thumbnails* ni otro tipo de exploración grafica ya que se tienen a utilizar de manera obligatoria los elementos que se crean con código en Maya, por esto se mantienen los colores y las tipografías.

Para un futuro se planifica hacer este tipo de exploraciones de otros elementos que irán programados de diferente manera.

High. La versión de alta fidelidad se toma como la versión prácticamente final de la interfaz. En la figura 15 se expone el modelo High con una serie de acciones y repercusiones del programa, en la figura 16 es como se pretende que se visualice Izi Rig en su versión final.

Figura 15



Representación del layout final de Izi rig y su flujo de uso.

Nota. En anexos se encuentra una versión escalada para su mejor visualización.

Figura 16



Representación de la interfaz final de Izi Rig.

Nota. Se debe tener en cuenta que este diseño se realiza antes de la implementación y se crea como la versión del programa.

Implementación de la Interfaz Gráfica

En el desarrollo de Izi Rig la etapa que conlleva más tiempo es la del *frontend*, concretamente su implementación. Esta etapa es crucial para el desarrollo de la herramienta, ya que son implementados sistemas como los markers o los trabajos para la selección de personajes.

Es el código creado en esta etapa el encargado de mover el código realizado en el *backend*.

Algunos Casos para Resaltar Durante el Proceso de la Implementación.

Creación de Ventanas. Maya no permite que las de las ventanas se creen sin una barra superior para el título de la ventana, lo cual afecta el diseño que se tiene planificado para la versión final. Ver la figura 17 tenemos dos ejemplos donde se evidencia la barra superior, la primera seria la normal y la segunda la única opción que Maya nos ofrece por medio de comandos básicos.

Figura 17



Referencia Visual de la barra de título.

Para entender por qué Maya no permite retirar la tabla se debe explicar un poco como el programa genera una ventana. Cada vez que Maya genera una ventana se corre un código específico (son mas de uno, pero para efectos de ejemplo solo se menciona uno), este código trae consigo una serie de argumentos que le aportan una serie de características a la ventana.

Autodesk facilita toda la información referente sobre sus comandos en help.autodesk.com. En la Figura 18 se puede ver un ejemplo de los argumentos que Maya nos permite usar para configurar una

ventana, sin embargo, en esta lista no se facilita un argumento que permita no generar la tabla superior de la manera que se pretende.

Tomar en cuenta que esta delimitación se aplica en muchos casos a lo largo de todo el proyecto.

Figura 18

Long name (short name)	Argument types	Properties
backgroundColor(bgcr)	[float, float, float]	3 4
The background color of the window. The arguments correspond to the red, green, and blue color components. Each component ranges in value from 0.0 to 1.0.		
closeCommand(cc)	script	3 4
Script executed after the window is closed.		
defineTemplate(dt)	string	3
Puts the command in a mode where any other flags and arguments are parsed and added to the command template specified in the argument. They will be used as default arguments in any subsequent invocations of the command when templateName is set as the current template.		
docTag(dtg)	string	3 4 1
Attach a tag to the window.		
dockCorner(dc)	[string, string]	3 4
Specifies which docking areas occupied the four different corners of the window. By default docking windows on the bottom or top will span the whole window. Use multiple instances of this flag to allow the left and right docking areas to occupy the corners. This method has two arguments: docking corner and docking area. Possible values for docking corner are "topLeft", "topRight", "bottomLeft", and "bottomRight". Possible values for docking area are "left", "right", "top", and "bottom".		
dockStation(ds)	boolean	3
When set this flag specifies that this window can contain other docked sub-windows.		
dockingLayout(dl)	string	3 4 1
When queried this flag will return a string holding the docking layout information. This string can be set when creating or editing a docking station to restore the previous docking layout. This string is a hexadecimal representation of a binary string and is not meant to be humanly readable, but can be saved and loaded using the optionVar command to restore layouts across sessions of Maya.		
exists(ex)	boolean	3
Returns whether the specified object exists or not. Other flags are ignored.		

Referencia visual de la lista de argumentos para el comando window.

Para Maya las ventanas no son solo una viñeta con información sobre esta, sino que son un compendio de columnas y filas donde se pueden colocar elementos gráficos, entre estos cabe resaltar los paneles, estos están vinculados a una cámara que recibe la información que uno le brinde al programa (modelo, *markers*, líneas de separación...)

Los paneles que Maya ofrece dentro de sus comandos son compatibles con el espacio de columnas y filas, sin embargo, estos no son configurables de la manera que se desea por su limitación de argumentos, en cambio se utilizó un comando que permite generar un panel al que se le añade una cámara y se puede construir totalmente desde cero. La limitación del uso de este comando es que no es compatible con las columnas y filas, solo funciona si es creado sobre una “forma”, espacios que a diferencia de las columnas o filas ordenan sus elementos por medio de coordenadas en un espacio 2D.

Creación de Botones. Maya no permite el uso de botones personalizados, pero permite agregar comandos a las imágenes, por lo que se implementaron imágenes y no botones en la mayoría de los “botones” de la interfaz.

Una vez que se soluciona el uso de botones personalizados, se procede a realizar los *markers* sobre la interfaz. Estos elementos son planificados en primera instancia como botones que posean una característica especial de *drag and drop* (arrastrar y soltar). Para lograr implementar un botón que pudiera ser arrastrado por el usuario se implementó un elemento ajeno a la interfaz el cual fue programado con características para que emule el funcionamiento de un botón y al mismo tiempo guarde una posición en el espacio.

Como parte crucial del proyecto se detalla más la problemática de los *markers* en el capítulo IV.

Creación de las Ventanas Fantasma. La creación de las ventanas que aparecen para dar una guía más acertada al usuario cuando este realiza el posicionamiento de los *markers* en las articulaciones del personaje. Se noto que en algunos personajes debido a su geometría se dificulta la visión del punto clave en el que se debe colocar el *markers*, es por eso por lo que se implementa esta ventana extra.

La ventana se le llama fantasma ya que solo aparece como guía cuando se tiene un *marker* seleccionado y se ha movido al menos una vez desde que fue seleccionado.

Figura 19



Referencia visual de la interfaz de Izi Rig con la ventana fantasma guía y la interfaz de Izi Rig con la ventana fantasma guía oculta.

Creación de Tablas de Contenido Extra. Izi Rig en su versión final contara tablas con contenido extra, dentro de este contenido se espera tener configuraciones para el usuario. En la versión alfa ya las tablas extras se encuentran implementadas, mas no funcionales, esto es debido a una delimitación del proyecto. El proyecto está realizado utilizando MayaPy 2.7 y para que la herramienta pueda almacenar y guardar información se deben implementar un tipo de archivo especial. json (notación javascriptObject), este es originalmente para Javascript y aunque a la fecha se convierte en un formato común se desconoce su funcionamiento con MayaPy 2.7. Dado que se planifica para la versión final, no se tiene planificado dentro del tiempo del proyecto y su desconocimiento por ser originalmente de un lenguaje distinto no se implementa.

Creación de Trabajos. Hay diferentes tipos de códigos que maya puede interpretar y realizar una acción, la mayoría de los códigos utilizados en Izi Rig corren únicamente cuando son activados por el usuario o un código superior.

Pero un gran avance en el *script* fue la implementación de códigos que funcionan contantemente en segundo plano esperando por eventos determinados, estos trabajos son más eficientes y dan una experiencia más agradable al usuario.

Se implemento el uso de trabajos en la selección del personaje, este trabajo le ordena a maya a catalogar, contar, medir, agrupar toda selección que el usuario haga, a su vez le indica al usuario el nombre de la selección que está haciendo por medio de la interfaz.

Etapas de Pruebas y Errores. Se debe desarrollar antes de salir al mercado. Como el proyecto de graduación está planificado para llegar hasta la etapa alfa, se contempla hasta la implementación y no se desarrolla la etapa de *testing*.

El proyecto originalmente está planificado para desarrollarse en un periodo de dos cuatrimestres. Es debido al tiempo que conlleva realizar pruebas contundentes, esperar los resultados de cada usuario, analizar estos resultados, catalogarlos, plantear una solución y realizar esta solución; que se planteó el

proyecto de graduación hasta la versión alfa de la herramienta. Se debe recordar que un software en etapa alfa esta realizado en su totalidad, solamente que no se ha sometido a una serie de pruebas.

Proceso de Desarrollo del *Rig*

Para Izi Rig la sección de *rig*, desde el proceso de planificación hasta su etapa de implementación es lo más importante del proyecto, puesto que el objetivo final es lograr un *rig* que sea útil y completamente usable por el usuario.

Requisitos Del *Rig*

Cuando se crea la idea de Izi Rig surgen dos ideas muy claras: generar el mejor rig posible y crear la mejor interfaz de un *autorig*. Estas dos ideas no son muy compatibles entre sí, por lo que se decide seccionar el programa principal en varios módulos especializados, con esto se logra facilitar el desarrollo del *rig* sin disminuir la capacidad de este.

En esta sección se explica todo lo que se espera que genere el programa como tal en el *rig*, no se centra en la interfaz ni como el usuario lo manipula, sino meramente el resultado que se puede obtener usando el programa.

Izi Rig Simple. Este módulo está enfocado en la facilidad a la hora de su utilización sin perder la capacidad en la generación de un *rig* eficiente. El módulo Simple está pensado para usuarios que necesitan un rig de manera rápida y sencilla y que a su vez no saben nada de riging.

Simple Human. Es el módulo más importante de Izi Rig, tanto la imagen principal del proyecto como la versión preliminar desarrollada en el proyecto de graduación. Está enfocado en la creación de un rig para personajes bípedos y humanoides.

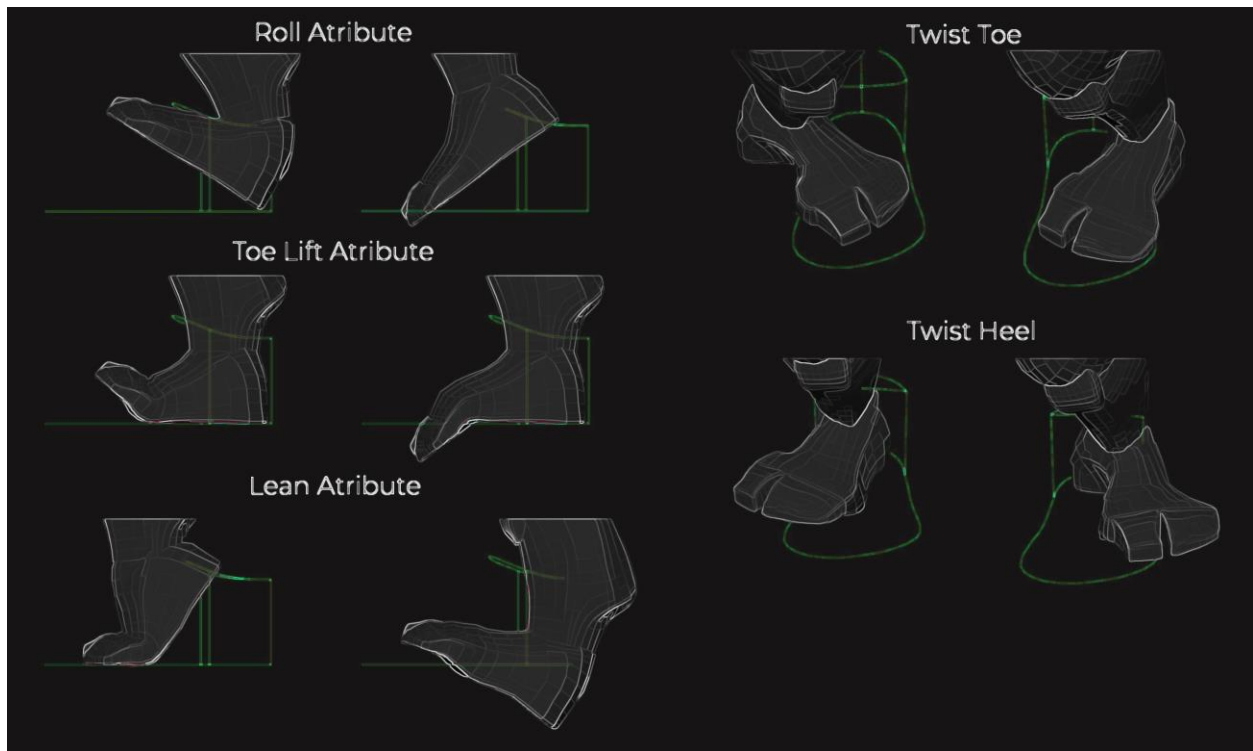
Características del *rig* simple human:

1. Piernas:

- 1.1. El máximo de piernas que se pueden generar es dos.

- 1.2. El *rig* se puede generar simétrico o asimétrico.
- 1.3. Una pierna puede generarse sin pie y sin rodilla.
- 1.4. La pierna se genera siempre con un sistema IK-FK
 - 1.4.1. Los controladores FK de la pierna son: *hip*, *knee*, *ankle* y *toe*. Los controladores solo rotan cada articulación.
 - 1.4.2. Para la pierna IK existe un controlador principal en el pie y otro para controlar el twist de la rodilla.
 - 1.4.3. Se genera un controlador únicamente para configuraciones de ambos sistemas, en este mismo se realiza el *switch* entre ambos sistemas.
- 1.5. Los atributos del sistema IK son:
 - 1.5.1. Rotación del tobillo en X
 - 1.5.2. Rotación de los dedos del pie (elevar y bajar dedos).
 - 1.5.3. Rotación del tobillo pivotando desde la base de los dedos del pie.
 - 1.5.4. Rotación en Y del pie con pivote en la punta del pie.
 - 1.5.5. Rotación en Y del pie con pivote en la base de los dedos del pie.
 - 1.5.6. Rotación en Y del pie con pivote en el talón. Ver Figura 20.
 - 1.5.7. Rotación en X del tobillo con pivote en el talón.
 - 1.5.8. Rotación en X del tobillo con pivote en la base de los dedos del pie.
 - 1.5.9. Rotación en X del tobillo con pivote en la punta del pie.

Figura 20



Referencia visual de la funcionalidad de los atributos del pie IK.

Nota: los tres atributos de rotación en X que se listan como: 1.5.7, 1.5.8 y 1.5.9, son extras y logran los mismos movimientos que se tienen en la imagen uno del roll atribute, y el lean attribute; pero estos son de manera independiente.

2. Brazos:

2.1. El máximo de Brazos que se pueden generar es dos.

2.2. El rig se puede generar simétrico o asimétrico.

2.3. Un brazo puede generarse sin mano, sin muñeca y sin codo.

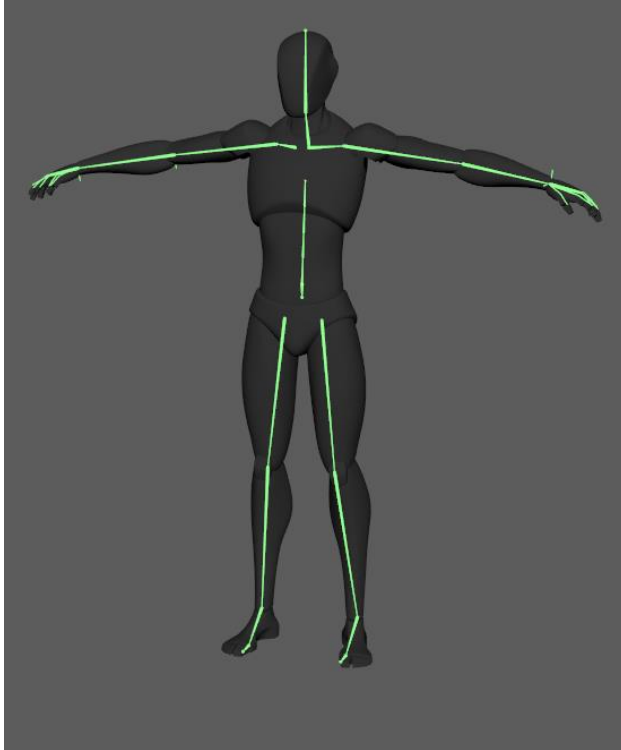
2.4. El brazo se genera siempre con un sistema IK-FK

2.4.1. Los controladores para el sistema FK son: *clavicle, shoulder, elbow, wrist*. Los controladores solo rotan cada articulación.

- 2.4.2. Para el sistema IK existen 2 controlares, el principal en la mano y un extra encargado del *twist* del codo.
- 2.4.3. El controlador FK de la clavícula es compartido entre ambos sistemas
- 2.4.4. Se genera un controlador únicamente para configuraciones de ambos sistemas, en este mismo se realiza el *switch* entre ambos sistemas.
- 2.5. El brazo tiene un sistema de *twist*.
- 2.6. Los dedos de la mano se pueden generar hasta un máximo de 5.
 - 2.6.1. Cada dedo se genera con tres controladores FK que únicamente pueden rotar la articulación.
- 3. Columna:
 - 3.1. Se genera una única columna vertebral.
 - 3.2. La columna usa un sistema híbrido FK-IK
 - 3.2.1. Se genera un controlador base y un controlador final para manipular el sistema IK.
Ambos controles pueden rotar y trasladar.
 - 3.2.2. Se genera un control FK, este control puede rotar.
 - 3.2.3. Para la pelvis y el cuello se generan dos controles FK de rotación.
 - 3.2.4. Se genera un controlador principal en el centro de gravedad del personaje (ubicado aproximadamente en el ombligo), este control puede rotar y trasladar.
- 4. Cabeza:
 - 4.1. La cabeza genera un único controlador FK que puede rotar.
 - 4.2. Se genera un controlador extra con configuraciones y atributos.
 - 4.3. La cabeza puede cambiar su punto de pivote y su espacio de rotación y jerarquía con respecto al cuerpo. Estos pueden ser: jerarquía directa con el cuello, con el centro de gravedad o con el suelo (esta última se toma como un vínculo a la nada).

5. Se generan dos controladores en la base del personaje: *move* y *root*, el primero puede ser rotado y trasladado, el *root* puede rotar, trasladar y escalar.

Figura 21



Referencia visual del rig final (solo de joints) generado por Izi Rig.

A partir de este punto se resumirán los siguientes requisitos, puesto que estos no entran en el proyecto final de graduación y su detallada redacción podría confundir o causar pérdida de la lectura.

Simple Animal. Módulo se especializa en animales, mantiene la identidad de lo simple, por lo que se divide en submódulos especializados en distintos tipos de morfologías.

1. Simple Bird, este submodulo esta enfocado en la creacion de aves, su fuerte esta en la generacion de extremidades aladas.
2. Simple Squid, este submodulo se enfoca en la creacion de tentaculos y aun esta en estudio la morfologia general que se pueda llegar a implementar.

Izi Rig Complex. Este módulo esta especializado en la creación de rigs mas complejos, dandole al usuario la capacidad de generar estructuras mucho mas complejas que las que permite Izi Rig Simple.

Izi Rig Complex no se enfoca en submodulos como lo hace Izi Rig simple, sino en usar extremidades entre si como si se contruyera una figura de LEGGO. Es por esto que Izi Rig Complex, aunque mantiene la estetica simple, esta enfocado en un público más especializado que quiere lograr un rig mas espezifico y dedicado.

Todas las extremidades que son generadas con Izi Rig Simple tambien pueden ser generadas por Izi Rig Complex, con la diferencia que esta no tiene limite de generacion.

Izi Rig Archive. Como su nombre indica, este apartado del programa esta enfocado en un a biblioteca virtual de apoyo para el usuario, tiene como objetivo facilitar el trabajo y agilizar el flujo del mismo.

Actualmente se define como un repositorio de Animaciones y poses que pueden ser aplicadas por el usario en cualquier rig que se haya generado con Izi Rig, estas animaciones pueden ser editadas por el usuario con la interfaz propietaria de Maya.

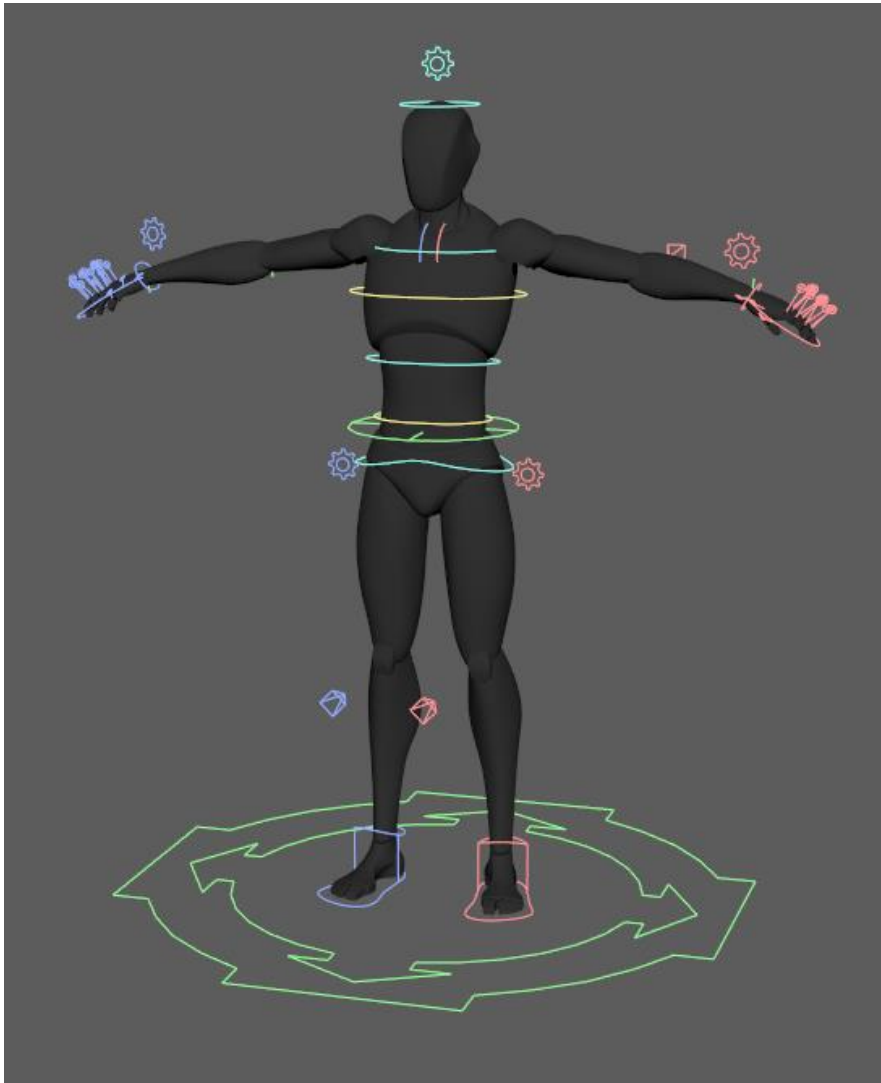
Este modulo dista mucho del *rig*, esta pensado como un apoyo a la herramienta.

Diseño del Rig

Apartado Estético. Se tuvo en cuenta como objetivo principal lo simple, con esto en mente se diseñan controladores minimalistas, se intenta evitar el uso de cajas y curvas exesivas que saturen la vista.

Para la elección de colores se mantienen siempre el estándar de colores de riging, pero se aplico una desaturación en la paleta de colores para generar una armonía más pastel.

Figura 22



Referencia visual del rig final generado por Izi Rig.

Nota: Se debe tomar en cuenta que, aunque Izi Rig tiene un sistema que mide la geometría para calcular el tamaño de las curvas para la generación de los controles, la topología puede tener especificaciones muy únicas, en estos casos puede que las curvas no se vean como en la referencia.

Apartado Técnico. Se focaliza en generar un rig con características básicas esenciales para el animador. Se obtiene un rig para personajes humanoides de dos piernas, dos brazos, una cabeza y un torzo; simétricos y con 5 dedos en cada mano.

Dentro de los sistemas de movimiento se logra mantener todos los mencionados en el apartado de requerimientos y no se efectúa ningún cambio para esta segunda etapa.

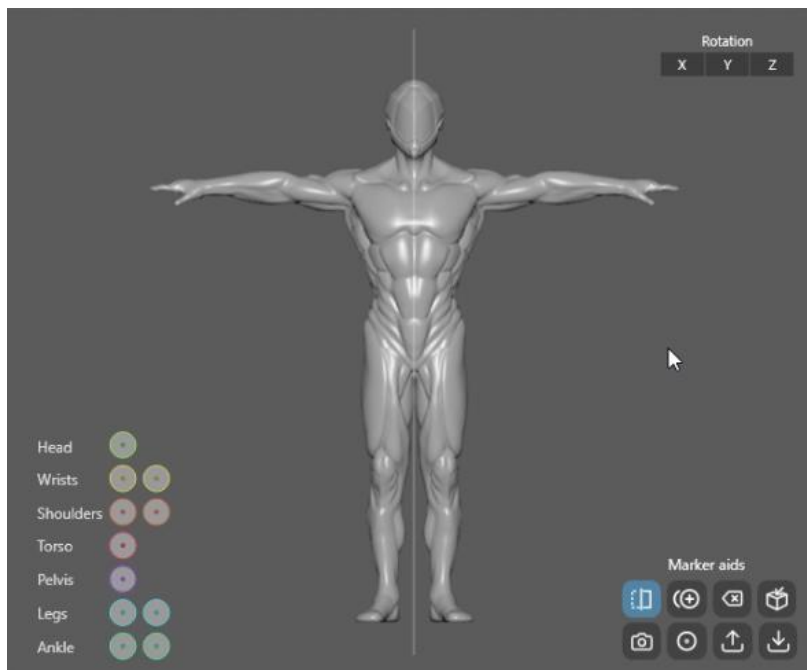
Implementación del Rig

Cabe recalcar que el código correspondiente a la creación de *rig* es una parte del *backend*, estos módulos no son usados de manera directa por el usuario, el programa dispone del uso de este código para lograr generar el *rig* dependiendo de las especificaciones que son brindadas por el *frontend*.

Localización de los Joints en la Posición Correcta

Se tuvo que crear todo un sistema propio, este sistema recopila las coordenadas del plano 2D que el usuario le aporta por medio de los *markers*, con esta información el sistema procede a medir la profundidad de la geometría del personaje en cada articulación, así obtiene la información para el eje faltante.

Figura 23



Referencia visual del concepto inicial de markers.

Nota: con la implementación de la interfaz surgen cambios notorios, pero se trata de mantener la esencia del diseño.

Izi Rig calcula la profundidad de la geometría tomando como referencia el punto X y Y, se emplea un sistema de rayos que son disparados desde -9999 en el eje Z hasta el 9999 en el eje Z, se toma el punto de impacto frontal y se almacena como dato “zmax”, se guarda el segundo punto de impacto como “zmin”, teniendo estos dos puntos de Z el programa calcula la distancia entre ambos y dependiendo de la articulación procede a almacenar el punto Z en la posición idónea.

En el caso de articulaciones que el usuario no indique las coordenadas X y Y Izi Rig triangula las distancias de las demás articulaciones y en base a estos datos rellena los datos faltantes. Para el caso del joint hip el script realiza la siguiente secuencia:

1. Localizar la distancia mínima en X de la cresta iliaca
2. Localizar la distancia mínima en Y de la entrepierna
3. Posicionar el hip en la diagonal que se crea entre estos dos puntos manteniendo el

Angulo creado entre la rodilla y el tobillo.

Creación del Esqueleto, Controles y Atributos

Con los datos de cada posición almacenados se procede a la generación del esqueleto principal (es en el esqueleto donde se crean los sistemas de movimiento del personaje), luego se crea el conjunto de controles, dentro de estos controles se añaden una serie de atributos necesarios para un mejor funcionamiento del *rig*.

Esta fase es relativamente sencilla de ejecutar, sumamente lineal y no varía nunca sin importar el modelo que se esté utilizando. Esta facilidad se debe a que Maya facilita al usuario una serie de comandos que el mismo programa utiliza para su interfaz propietaria.

Para dar una breve explicación de la creación y utilización de estos comandos se adjuntan una serie de estos, sin embargo, en esta sección no se profundiza la estructura del código utilizado, para mayor explicación ir al punto 3.3.

Cuando un usuario da clic en *Create Joints* Maya corre de manera inmediata:

```
JointTool
```

Este nos permite usar la herramienta de creación de *joints*, cuando el usuario empleando la herramienta de clic izquierdo en el espacio (supongamos que en el punto central del mundo) Maya corre otro código:

```
joint -p 0 0 0
```

Este código le indica a Maya que se debe crear un *joint* en la posición 0, 0, 0 (el centro del mundo). El lenguaje que Maya emplea para sus comandos se llama MEL, Izi Rig fue realizado utilizando Python, por lo que el comando para crear un *joint* se debe traducir, obteniendo:

```
cmds.joint(p=(0,0,0))
```

Con este código que maya brinda se puede generar un *joint* de la misma manera que se genera utilizando la herramienta, con la diferencia que los atributos hay que darlos como información directa y no por medio de la interfaz de usuario. En el caso de que se quiera crear un *joint* de nombre "elbow_01", con un radio de 0.5 y en las coordenadas (0, 15, 23) se usaría:

```
cmds.joint(n="elbow_01", p=(0, 15, 23), rad=0.5)
```

Es por esto por lo que el proceso de creación del *rig* se asemeja en gran manera al proceso de *rig* manual, solamente que este se da de manera automatizada. Por lo que realmente no se requieren de procesos extra o procesos complejos para la creación del *rig*, por lo tanto, se listan los procesos necesarios del *rig* sin entrar en una explicación detallada de cada uno.

Para cadena de Joints, Constrains y Conexiones;

1. Izi rig toma los datos de cada articulación para generar cada cadena de *joints* necesaria del *rig*, se duplican las cadenas para los sistemas IK-FK.
2. Se crean una serie de *Parents constrains* entre la cadena principal y las cadenas IK-FK.

3. Se crean grupos que funcionaran como *null* para cada curva, al mismo tiempo se crean las curvas en la localización de la articulación, con su rotación correcta y sus atributos necesarios configurados de manera correcta.
4. Se finaliza el *rig* organizando el *outliner* de una manera en la que cada grupo de curvas es independiente a los grupos de los *joints* a excepción de las curvas que están emparentadas de manera directa con el *joint*. Se ocultan todos los *shapes* y nodos que el usuario no debe ser capas de editar. Se crea un set con todos los *joints* que están ligados a los pesos del personaje.

Documentación

Manual de Instalación



IZI RIG INSTALLATION GUIDE

DESCRIPTION

Auto rig script for Maya 2020.

REQUIREMENTS

This script has been made to work with Maya 2020 but may work with other versions as well.

INSTALL

1. Quit Maya if it is running.
2. Un-zip the contents of the scripts to your local sourceable script path for yourself. Typically something like:

C:\Documents and Settings\YOURNAME\My Documents\maya\scripts\

3. Start Maya.

Copyright ©2021 Zúñiga Sánchez Alberto All Rights Reserved



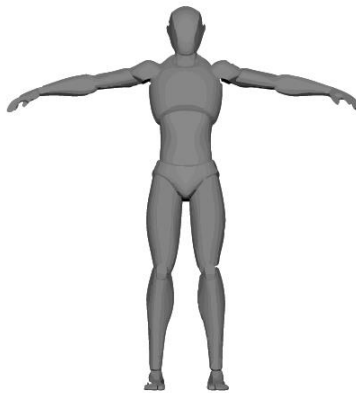
USER GUIDE

Character Preparation

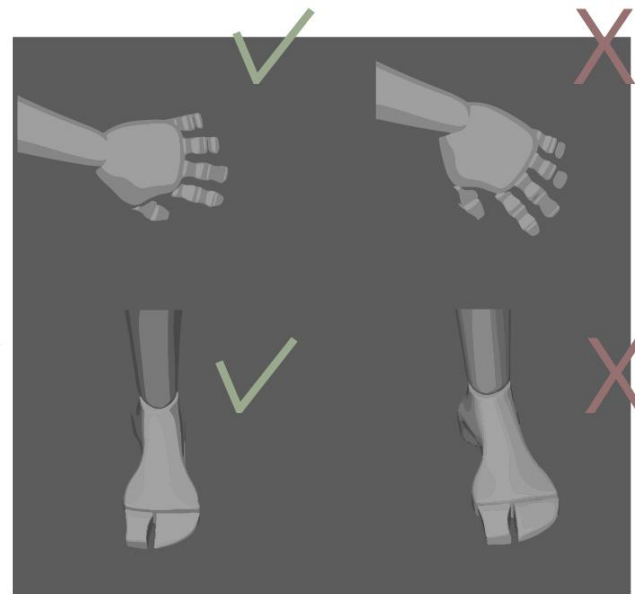
To have a better script performance, there is a series of recommendations in order to prepare the character that you will apply the rig to.

- 1- Izi Rig only reads characters that are in a T-Pose or an A-Pose.
- 2- It is recommended to pose the hand towards the X axis in Maya.

- 3- The position of the feet need to be in a straight position in the Y axis for it to have a correct rotation.



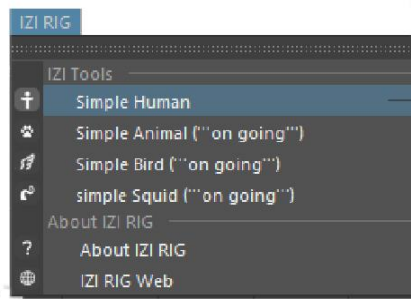
Visualización de la pose recomendada



Use of the Tool

With the character ready to rig: :

1- Open the Izi Rig Interface.

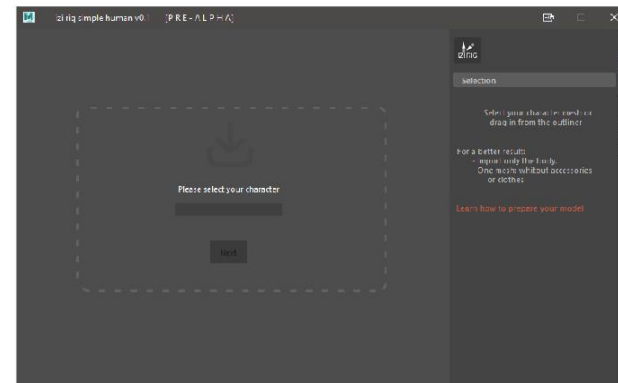


Specialized script for the humanoid biped rigs.

2- Select the character

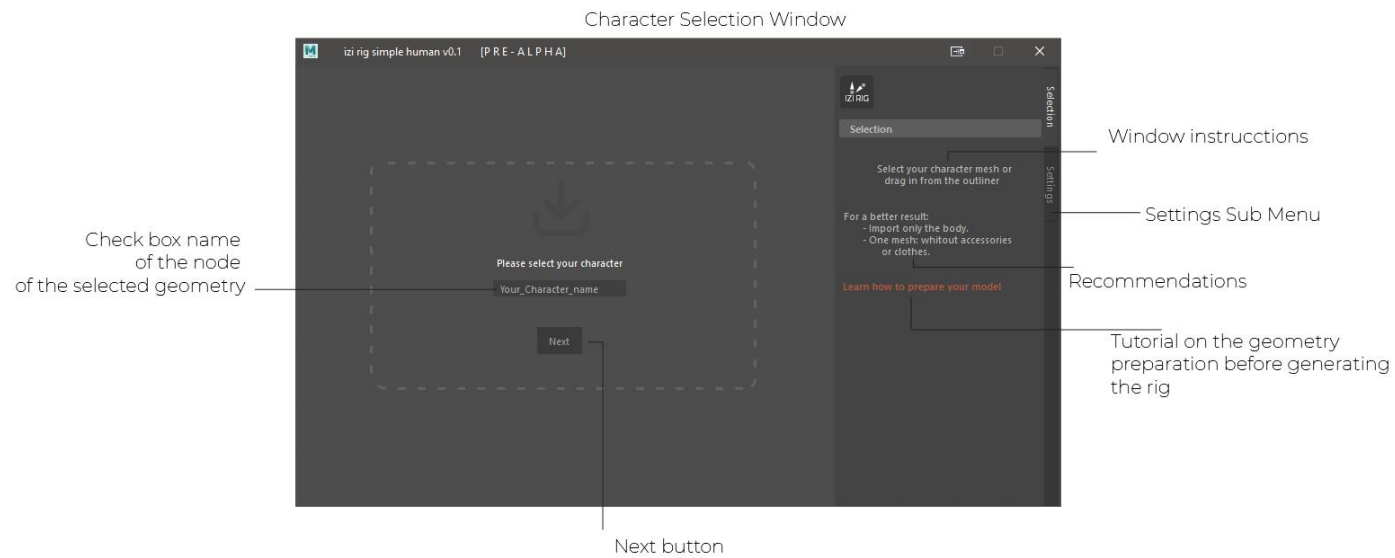
This step can be done by selecting the node directly from the outliner or by selecting the geometry from the camera in Maya.

Character selection window



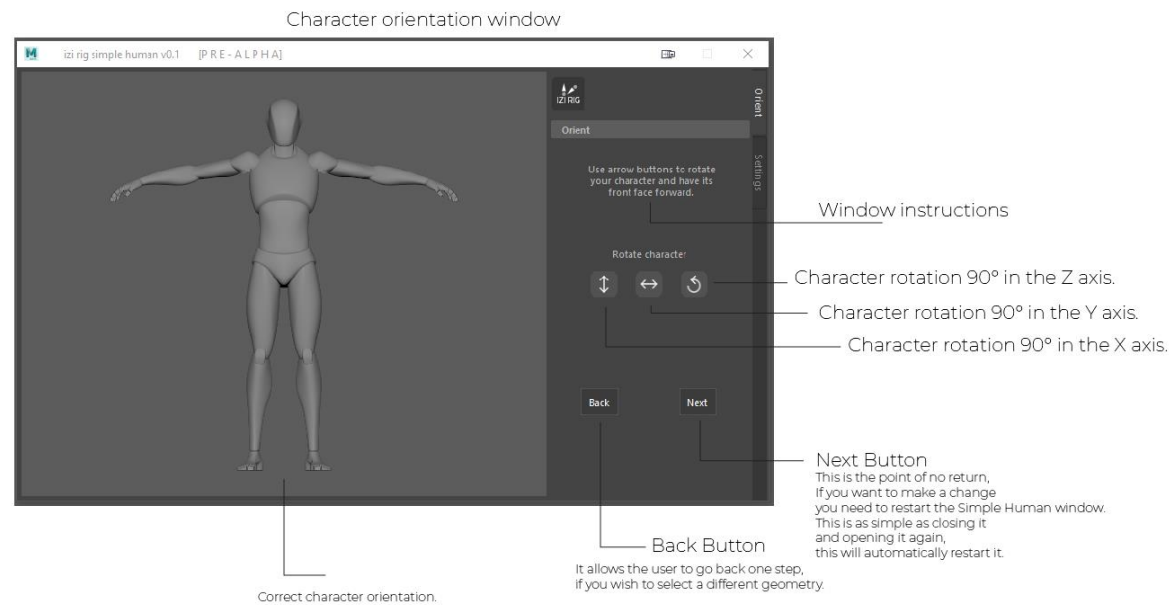
It is highly recommended that the character is only one piece of geometry.
Izi Rig automatically will combine the character if it is made from different meshes.

Use of the Tool



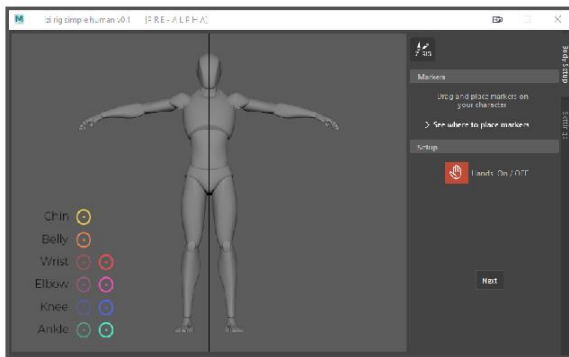
Use of the Tool

3- Correct character orientation



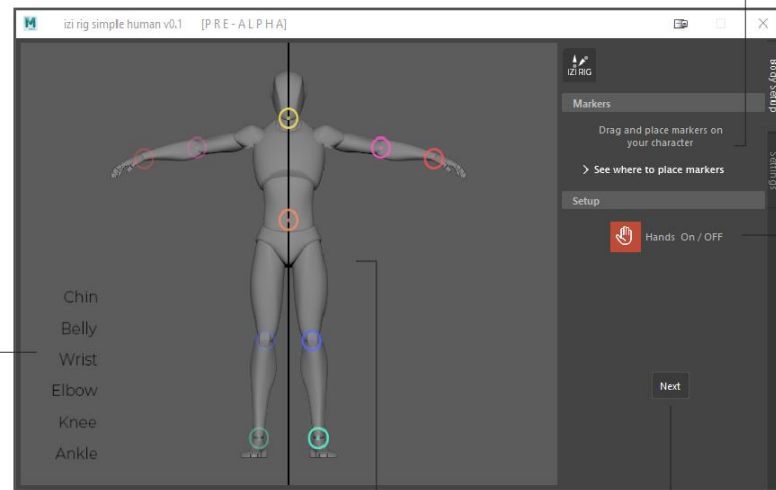
Use of the Tool

4- Placement of the markers on the joints of the model



Original marker position

Marker Sub Menu

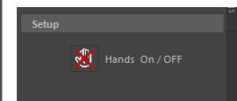


Correct Arrangement of Markers



Window with instructions,
guide of marker
placement on character

Button in case that no hands
are needed in the character.

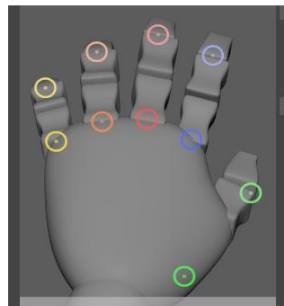


Next button

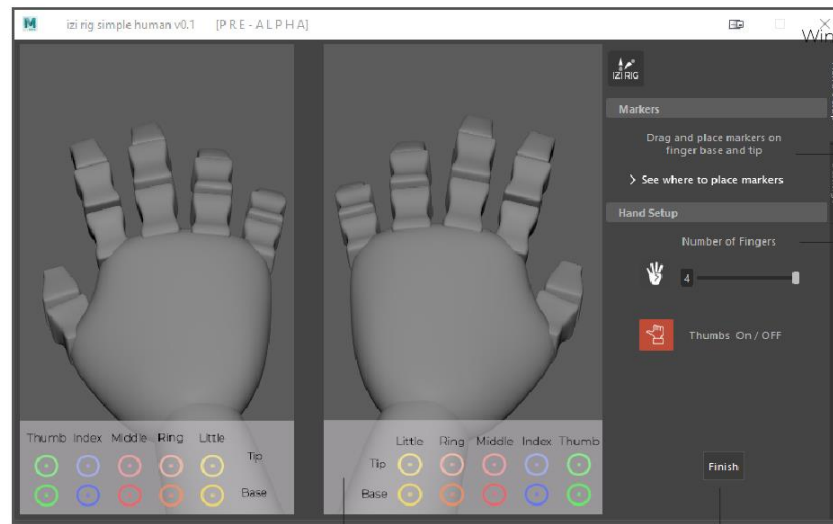
Use of the Tool

If you turned off the hand button skip this step.

5- Marker placement on the hand joints



Correct marker arrangement



Marker menu

Button to finalize process

Window with instructions,
Marker placement
on hand guide



Special Setting for hand Sub menu

The user can choose the number of fingers desired in total from a minimum of 1 to a maximum of 4.

The thumb is a finger done apart. You can also choose if you want to have the thumb or not.



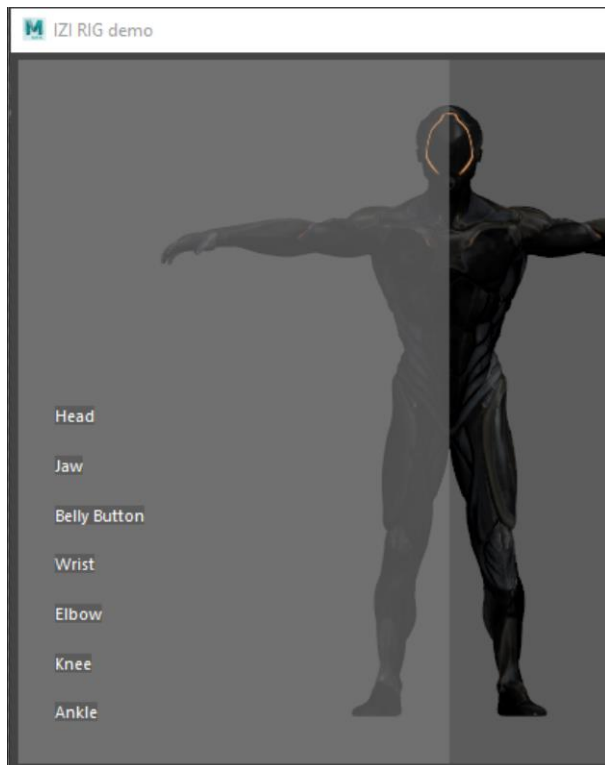
CAPITULO IV

Conclusiones y Recomendaciones

Problemas Durante la Implementación de la Interfaz

Trasparencias de OpenGL. Durante el proceso de implementación de la interfaz, concretamente el código referente a los *markers* se chocó con una limitación que Maya tiene como software, concretamente el uso de OpenGL, y es que lenguaje no soporta transparencias en la interfaz, lo cual ocasiona un gran retraso en el cronograma de Izi Rig.

Figura 24



Referencia visual de la falta de transparencia en el texto sobre el editor.

La primera solución que se implementa es usar colores planos para el fondo del texto y descartar el uso de alphas en las imágenes, esta solución se descarta ya que, si el usuario cambia el fondo de Maya, el color del texto es delatado. También se descubre que Maya de manera automática prioriza el elemento

de UI que se tenga seleccionado y lo posiciona como elemento frontal del UI, por lo tanto, si el usuario falla al seleccionar el *marker* y da clic en la interfaz esta automáticamente estaría superpuesta a los *markers*, ocultándolos así de la interfaz.

Debido a todas las complicaciones que sufre el uso de elementos de interfaz, se procede a implementar elementos en el espacio 3D. Con la implementación de elementos ajenos a la interfaz se logra solucionar ambos problemas, se logra el uso de transparencias en los textos, botones y *markers*; también se logra la manipulación idónea de estos elementos.

Manipulación de los Markers

A raíz del uso de elementos 3D como interfaz surgen otra serie de problemas:

- 1- El usuario al seleccionar un *marker* automáticamente genera el gizmo de Maya.
- 2- Maya por defecto remarca el elemento que se tiene seleccionado

Para evitar que Maya utilice el resaltado de elementos de creo el siguiente código:

```
if cmds.displayPref(q=True, wsa=True) == "full":  
    cmds.displayPref(wsa="none")  
else:  
    cmds.displayPref(wsa="full")
```

Este código primero revisa si el resaltado de elementos esta activo, si este lo está, lo apaga. Así es como Izi Rig logra que los *markers* no resalten al ser seleccionados y aparenten ser elementos nativos de la interfaz.

Creación de herramientas extra (*drag and drop*). Para poder trasladar los markers Maya por defecto selecciona la herramienta de movimiento (*move tool*), dicha herramienta es notable cuando se utiliza y no se puede ocultar el gizmo de ninguna manera amable para el programa ni el usuario.

La herramienta *move* permite arrastrar un elemento, pero para eso el usuario debe seleccionar el elemento, esto se logra dando un clic completo (oprimir y soltar), luego debe dar un clic y mantener apretado para arrastrar el elemento. Este sistema de selección es anticuado y choca con los principios de sencillez que Izi Rig implementa, es por eso por lo que se creó una herramienta específica para esta función.

La herramienta que se crea no solo permite al usuario mover de manera fácil y sin molestias visuales los *markers*, dicha herramienta también corre una serie de comandos en paralelo que activan y desactivan la ventana fantasma guía, al mismo tiempo esta herramienta maneja la simetría de los *markers*.

Conclusiones la Interfaz

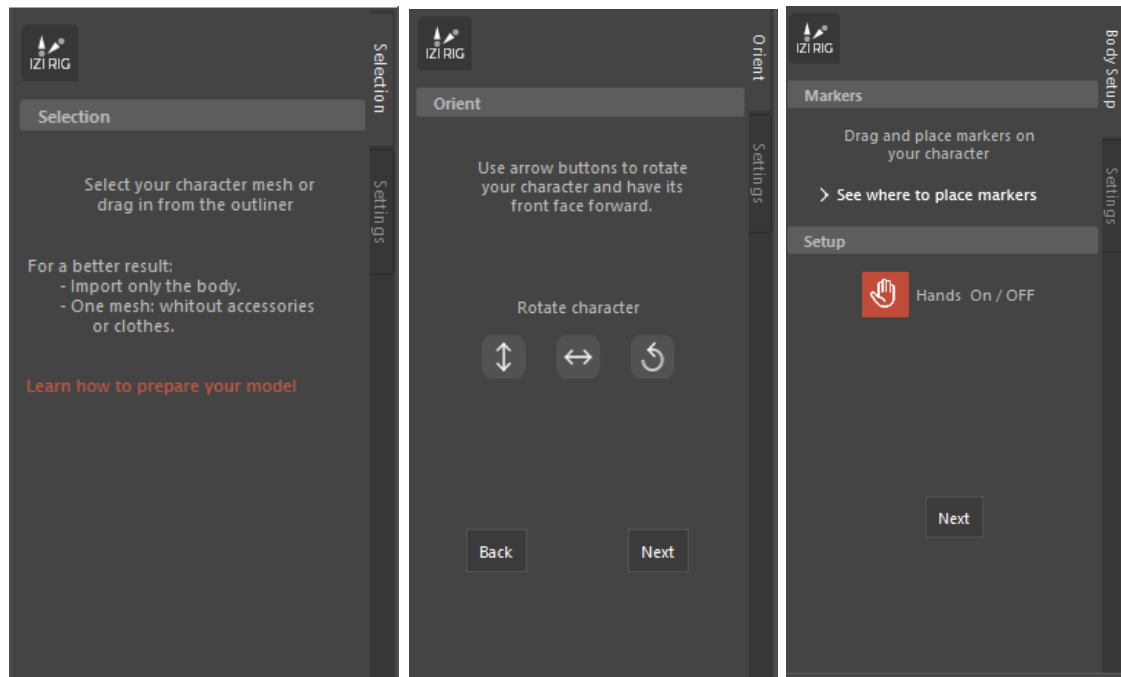
El proceso de la creación de la interfaz de Izi Rig es por lejos el más tardado y que más problemas presenta a la hora de su implementación, pero a su vez ha sido el proceso mediante el que el estudiante ha adquirido más conocimiento, también se reconoce que aunque surgen bastantes problemas en la implementación de la interfaz su calidad en vez de disminuir aumenta, es el caso de la ventana fantasma, la cual no estaba planificada en sus inicios, pero se vio viable su implementación dentro del proyecto.

Se logra de manera exitosa generar una interfaz amigable con el usuario, se respetan elementos como los *markers* y la interfaz de rotación del personaje. Se mejora la interfaz de selección de personaje con el uso de los *Jobs* de Maya en donde con un solo clic el personaje está listo para trabajar.

Se logra mejorar el diseño del menú lateral editando los botones que Maya trae por defecto y haciendo más fácil de leer el UI. Ver Figura 24. Se logra implementar un sistema que calcula automáticamente el ángulo de los brazos del personaje y lee si este se encuentra en pose "T" o en pose "A". Este último sistema fue lo último que se desarrolló para la interfaz de Izi Rig y no se implementa en la versión Alpha, ya que, aunque funcione de manera eficiente y sin errores, el *backend* no soporta las

manos en pose A, esto debido a la infinita posibilidad de posiciones en la que las manos pueden estar dispuestas.

Figura 24



Referencia visual de la interfaz final de Izi Rig.

Se logra generar el Rig de las manos, este ultimo debido a la complejidad de la geometría es el sistema que es mas complejo dentro de Izi Rig, no solo en el nivel de interfaz sino como de backend.

Conclusiones del Rig

Como se expone a lo largo del documento, la creación del rig resulta ser la zona pacifica del proyecto, hasta que se habla de la generación de las manos.

Cuando se implementan las manos en Izi Rig surge una lista bastante extensa de problemas, pero para efectos de proyecto solo se enumeran los problemas más importantes.

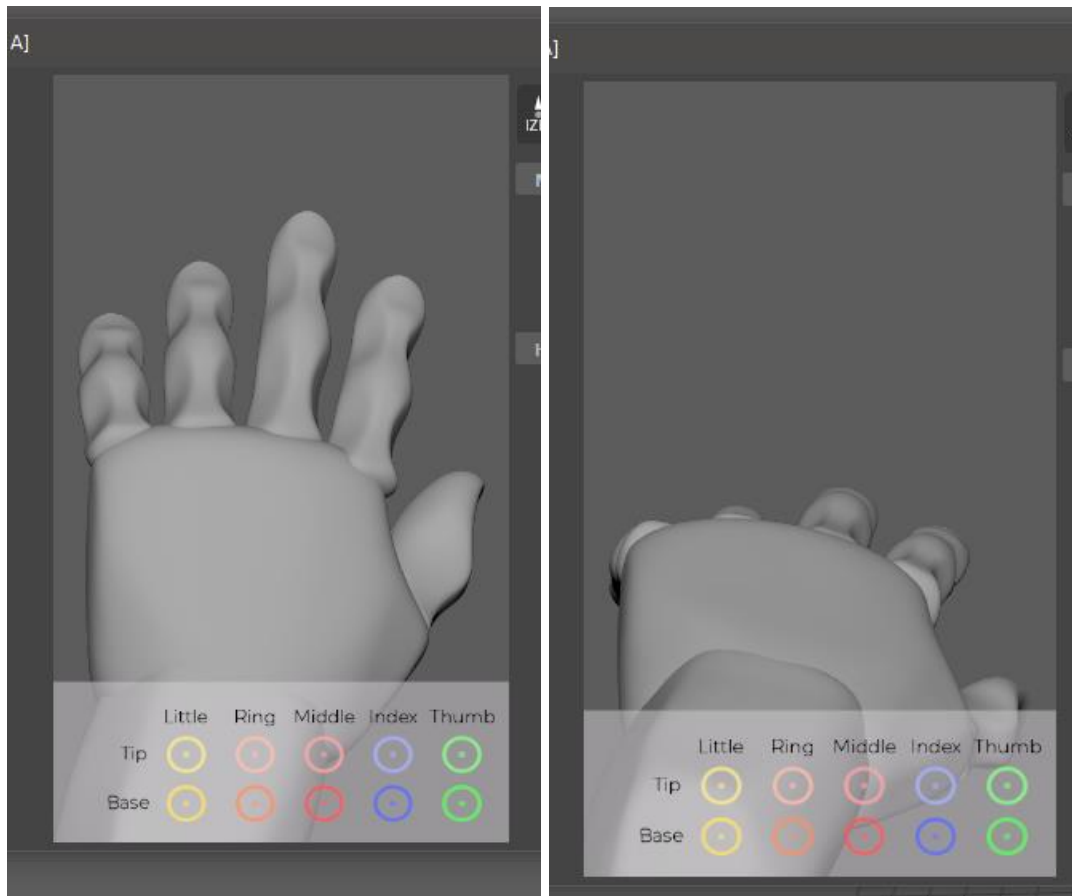
Calcular la Posición de la Muñeca. Cuando el Usuario selecciona el personaje, el trabajo del backend mide la altura, el ancho y el fondo de este, así mismo revisa si el personaje está en la posición

correcta para ser rigeadado, si no lo está, Izi Rig posiciona a la persona en las coordenadas (0, 0, 0), el Y positivo, para asegurar que no ha geometría en ejes negativos y posiciona el pivote del personaje en (0, 0, 0). Todo esto es posible porque sabemos que el punto mas bajo de la geometría es la base del personaje y que este debe estar en 0, pero con la muñeca esto no se sabe.

Izi Rig implementa un sistema de etapas, en la primera etapa de markers el usuario da la posición de X y Y de la muñeca, Izi Rig calcula la profundidad de esta y con esos datos genera la cámara en la posición ideal para que el usuario pueda ver la mano de manera correcta, pero hay factores que se deben tomar en cuenta:

1. La cámara no puede estar siempre en un solo eje, ya que si el personaje esta en pose "A" los dedos no se observan de la manera correcta desde una vista idónea para un personaje en pose "T".

Figura 25



Referencia visual de la cámara de la mano sin usar el algoritmo para calcular el ángulo del brazo.

2. Se debe tener en cuenta que la mano tiende a ser mas larga por lo tanto si se genera la cámara apuntando a la muñeca, la punta de los dedos puede quedar amputada, por lo mismo se crea la fórmula:

```
(var.lWristPosition[0] - (var.meshHeight * 0.02876), 9998,  
var.lWristPosition[2] + (var.meshHeight * -0.00155))
```

Donde `var.lWristPosition[0]` es la posición de la muñeca en el eje X, `var.meshHeight` es la altura del personaje desde el 0 del mundo. Con esta formula se logra posicionar la cámara de mejor manera en el eje X. (todo esto para casos de personajes en pose “T”).

3. Se debe tener en cuenta que el ancho de la mano también influye, por lo que se utiliza el sistema de rayos para calcular los puntos máximos de la geometría de los dedos y saber los márgenes que debe tener la cámara (*orthographic width*).
4. También de debe tener en cuenta la rotación que puede tener la muñeca, este problema causa una inclinación en la cámara resultando en un error visual donde los *markers* se pierden. Ver Figura 25.

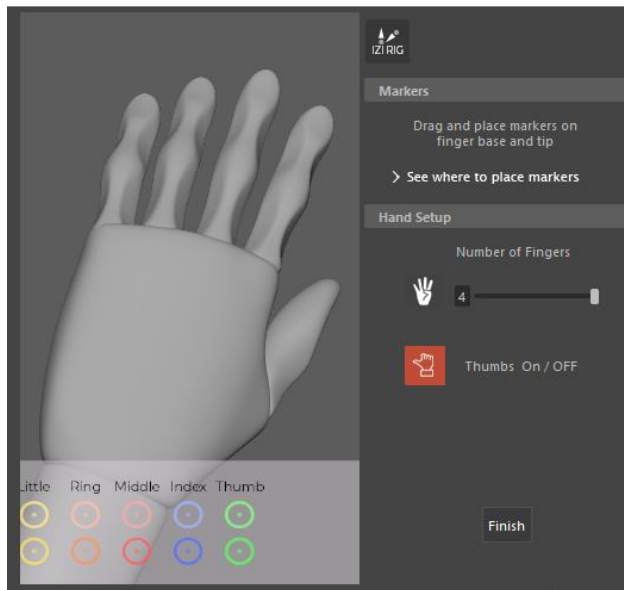
Conclusiones

Se concluye que para la versión final de Izi Rig se debe hacer una reestructuración de la interfaz de la mano, puesto que el numero de variables es tal que nubla la interfaz y la vuelve poco eficiente, las posibles soluciones son:

Generar el *rig* de las manos de manera arbitraria y evitar un paso extra en la interfaz.

Utilizar el entorno 3D y *markers* 3D en este paso para que el usuario pueda de manera mas idónea posicionar los dedos sin importar el tipo de pose, ángulo de inclinación de los dedos, ángulo de inclinación de la muñeca, relajación de los dedos...

Figura 25



Referencia visual de la cámara de la mano cuando se tiene una inclinación de muñeca en el eje Y.

Se recomienda emplear el sistema de medición de profundidad para crear un mejor mapeo de la topología del personaje y de esa manera solventar aún mejor la generación de las curvas de los controles.

Se tiene en mente generar una interfaz donde el usuario pueda elegir si ver una mayor cantidad de *markers* para así tener una mejor localización del *joint*.

Se visualiza la herramienta como un completo éxito dado que su funcionamiento es eficiente (rápido), se logra la simplicidad en el uso y al ser dentro de la interfaz propia del programa el *rig* que se genera esta completamente listo para ser usado o editado en el mismo instante que se genera.

CAPITULO V

Bibliografía

Bear Studios. (2019). Bear. Recuperado 10 de febrero de 2020, de <https://www.bearstudios.co.uk/what-we-do>

Blaabjerg, B. (s. f.). Bjørn Blaabjerg. Recuperado 24 de marzo de 2020, de <https://blaabjergb.com/>

Blendermarket. (2017). Blendermarket. Recuperado 15 de febrero de 2020, de <https://blendermarket.com/products/auto-rig-pro?ref=46>

Dustin Nelson. (2016, marzo 30). *Rapid Rig Modular: Modules Tutorial* [Archivo de vídeo]. Recuperado de https://www.youtube.com/watch?v=sX_iUaE70iQ

Highend3d. (s. f.). Rapid Rig. Recuperado 6 de marzo de 2020, de <https://www.bearstudios.co.uk/store/rigomator>

Lin, X. (2020, abril 15). *Component Based Procedure Auto Rig* [Archivo de vídeo]. Recuperado de <https://vimeo.com/342926245>

LinkedIn. (s. f.). Dustin Nelson. Recuperado 24 de marzo de 2020, de <https://www.linkedin.com/in/dustin-nelson-24a61421/?originalSubdomain=ca>

LinkedIn. (s. f.-b). Xiong Lin. Recuperado 24 de marzo de 2020, de <https://www.linkedin.com/in/xiong6744/>

Veber, L. (2017). Lucas Veber. Recuperado 2 de marzo de 2020, de http://www.lucky3d.fr/tools_auto_rig_pro.php?page=tools

D. L. Parnas, P. C. Clements and D. M. Weiss, "The Modular Structure of Complex Systems," in *IEEE Transactions on Software Engineering*, vol. SE-11, no. 3, pp. 259-266, March 1985, doi: 10.1109/TSE.1985.232209.

Anexos

Anexo 1

EXPLORACIÓN DEL LOGO



Exploración del logo 01

Anexo 2

LOGO FINAL

FONT:

Montserrat Thin #494949

Montserrat Thin #FFFFFF



#494949
Opacity: 100%



#494949
Opacity: 55%



#FFFFFF
Opacity: 100%



#FFFFFF
Opacity: 55%



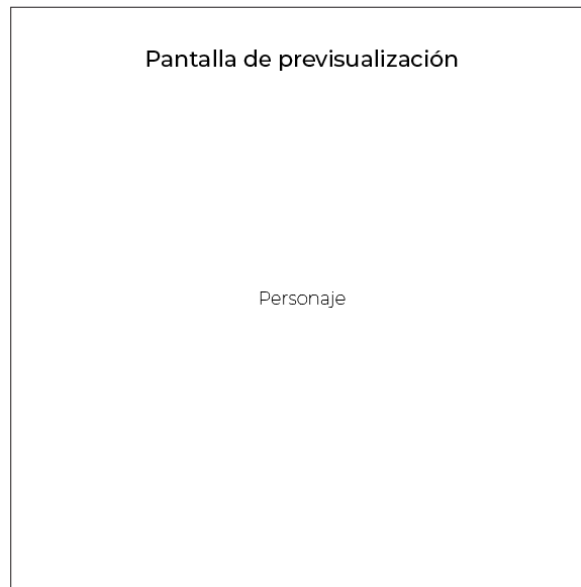
Fondo Claro



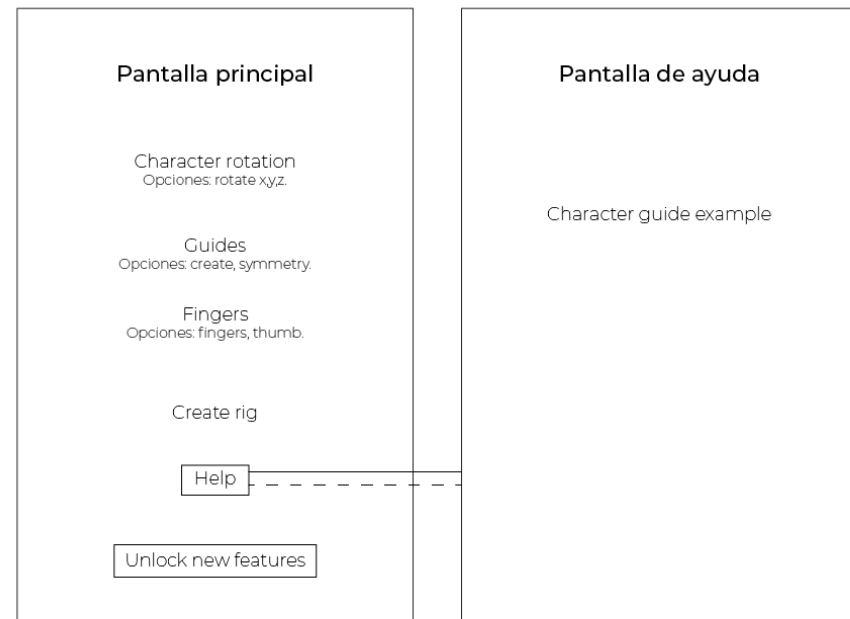
Fondo Oscuro

Exploración del logo 02

Anexo 3



* Siempre abierto



Simbología



No es un botón, quiere decir que este set de información lleva a un segundo set de información.

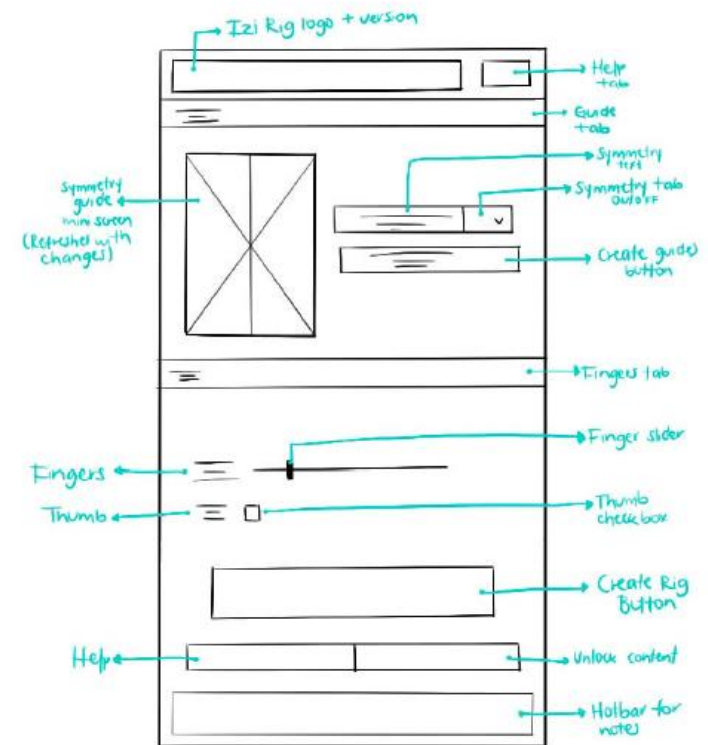
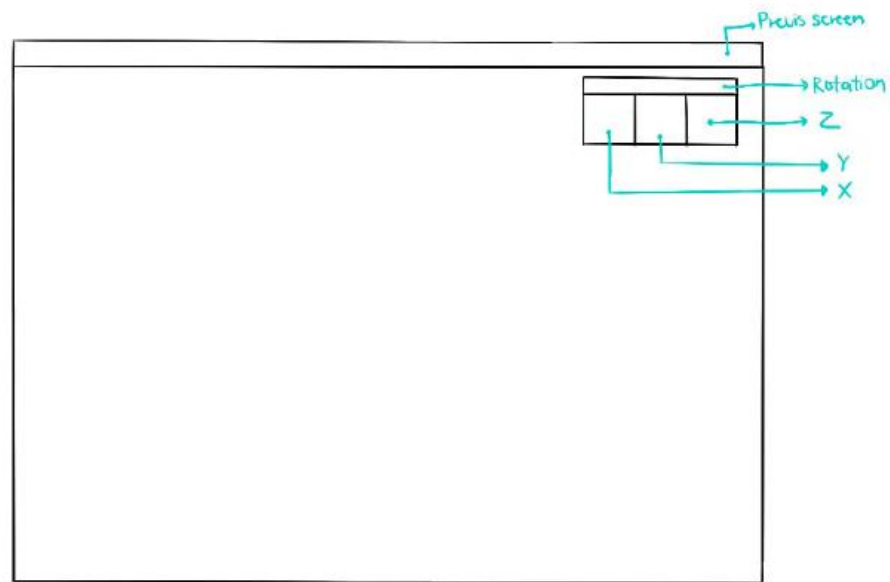


Camino de información (se conecta con, lleva a...)

Camino de regreso de información (se regresa a _ desde _)

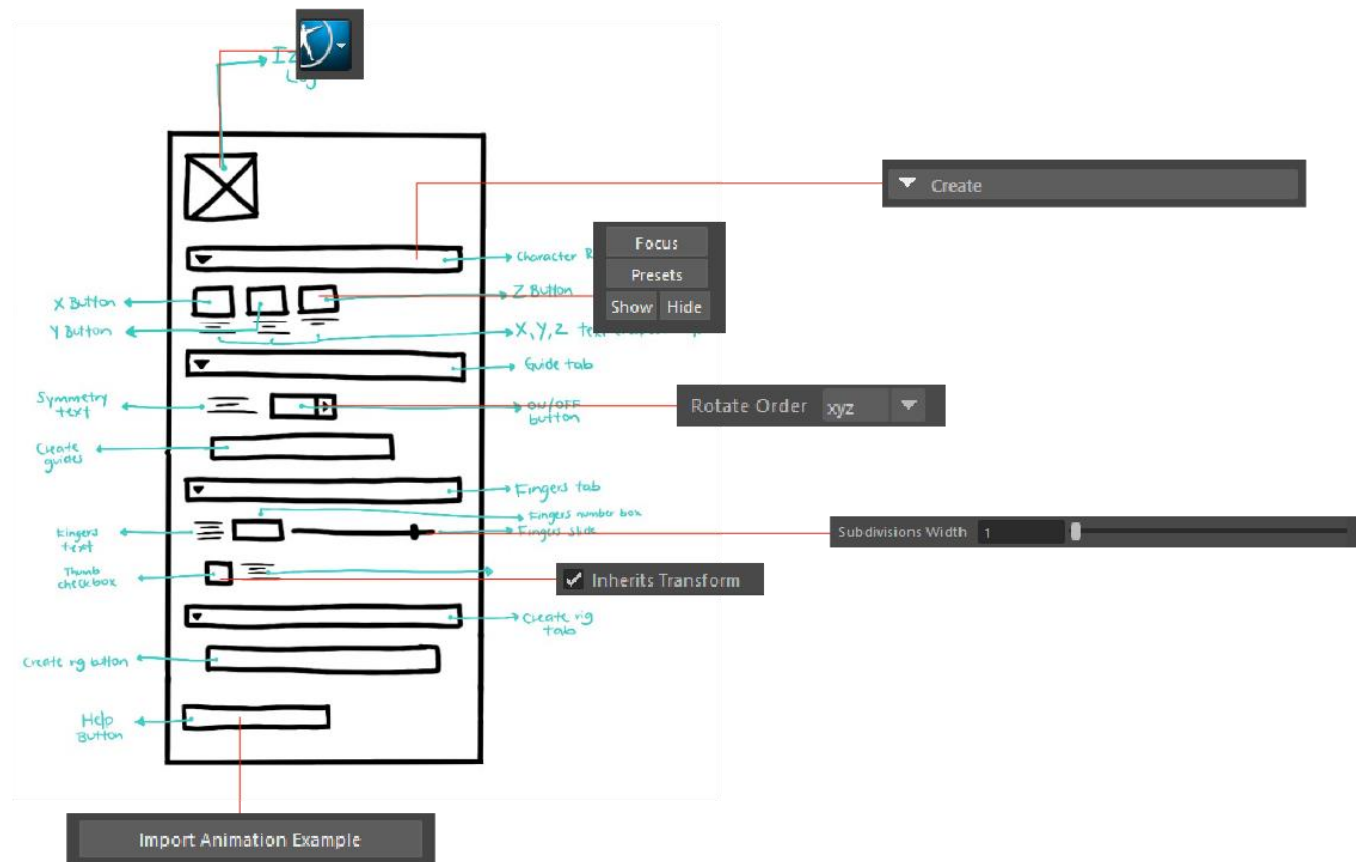
Site Map 01

Anexo 4



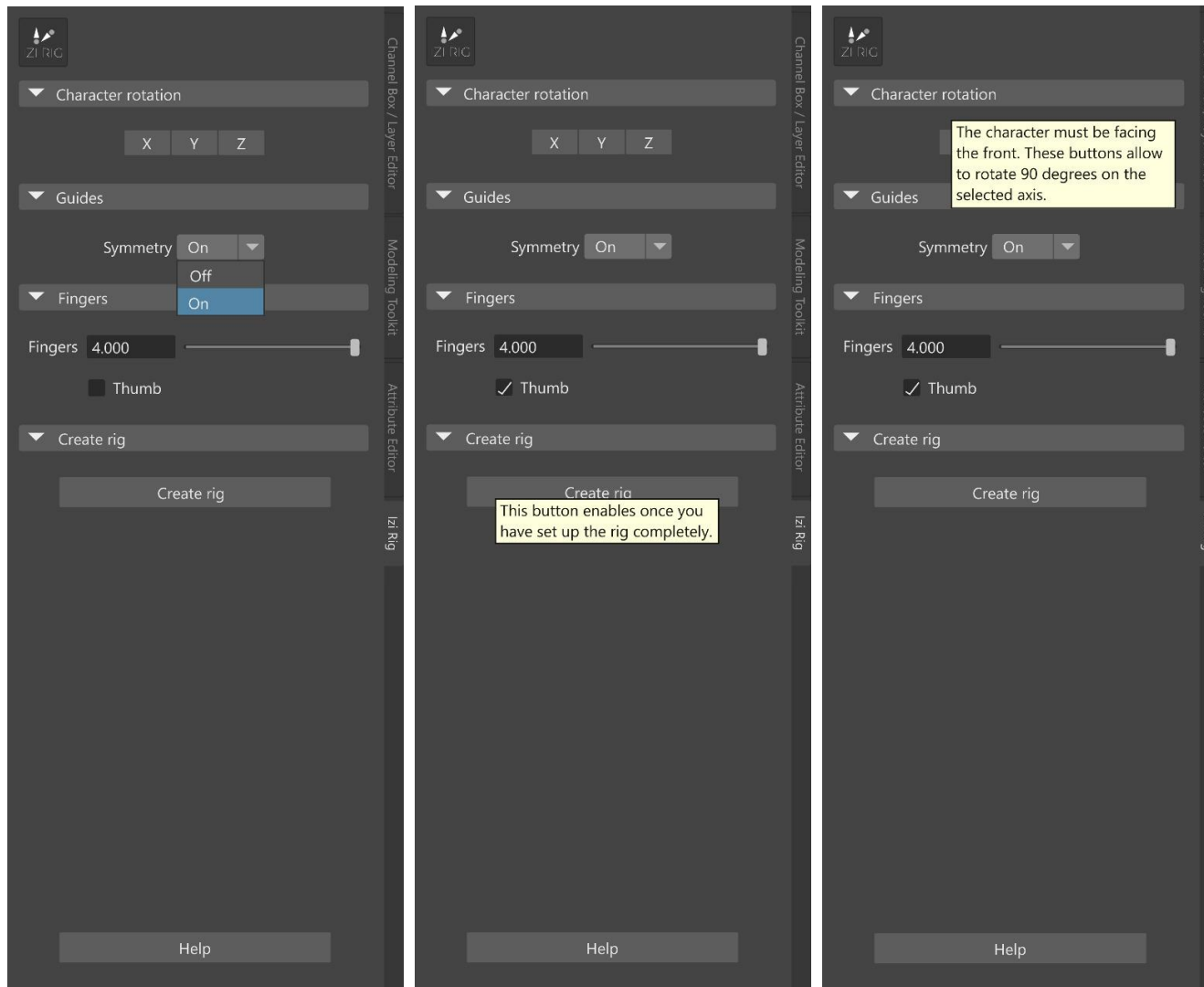
Wireframe lo-fi

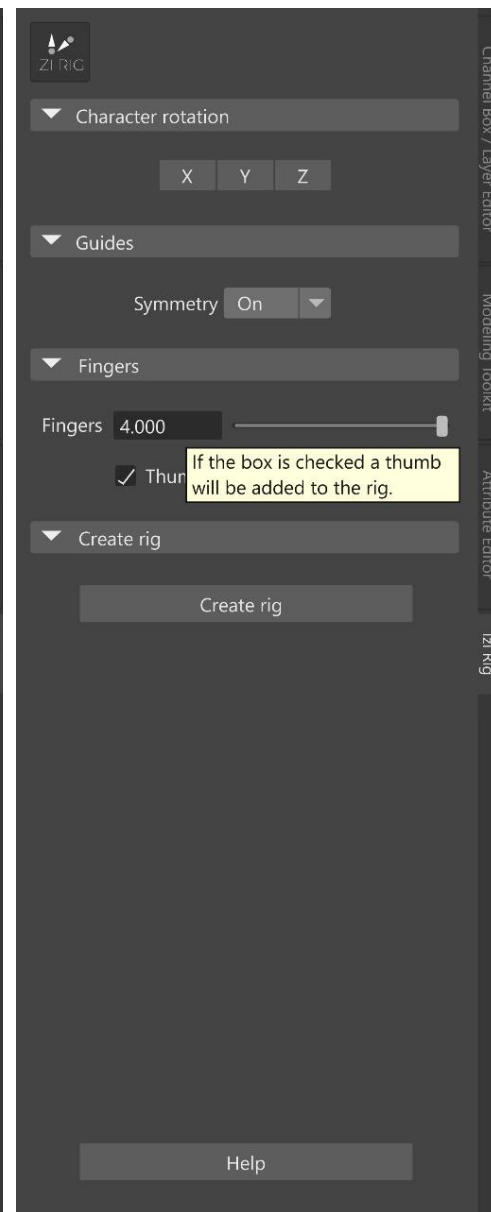
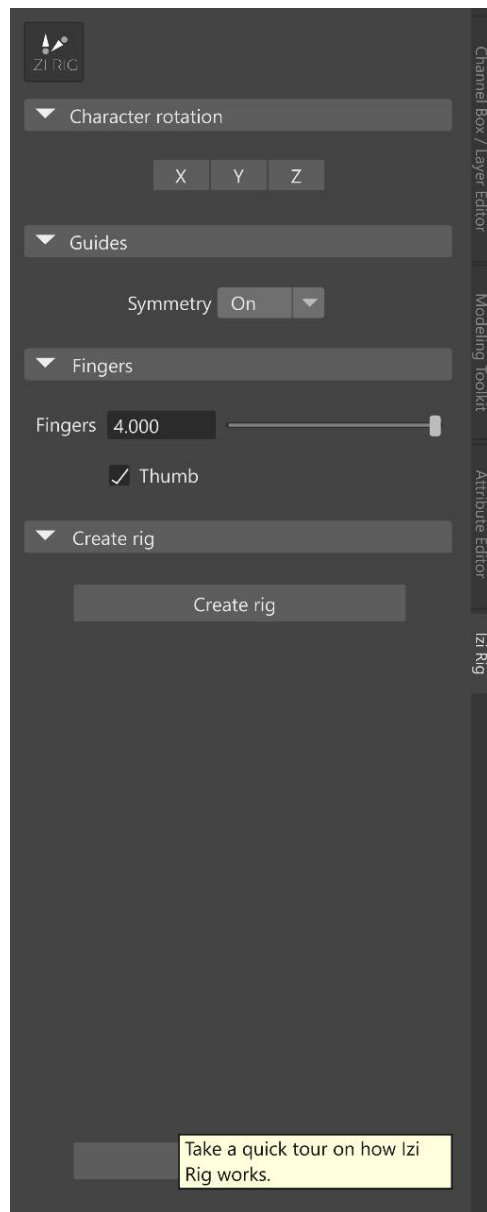
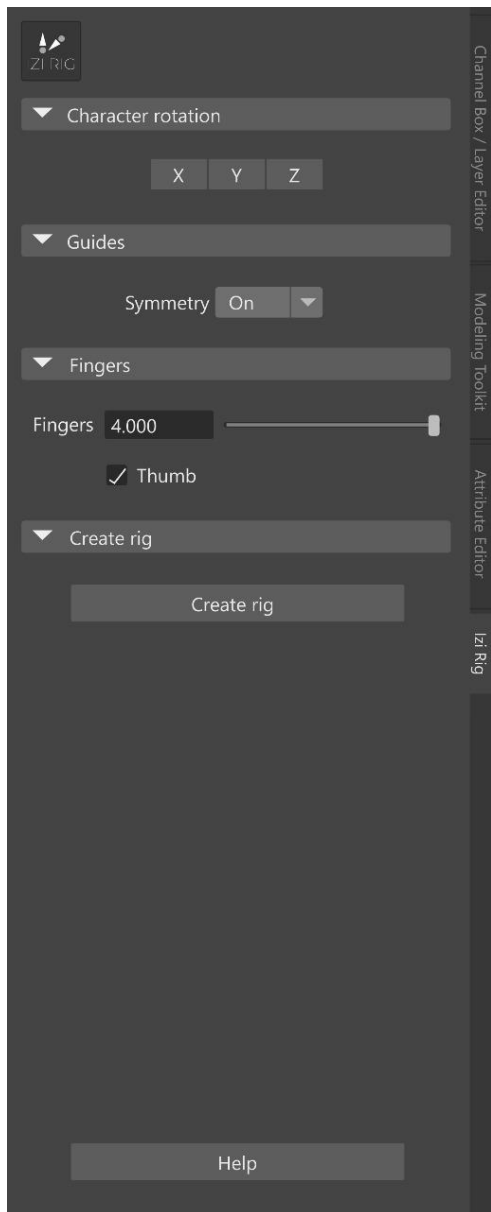
Anexo 5

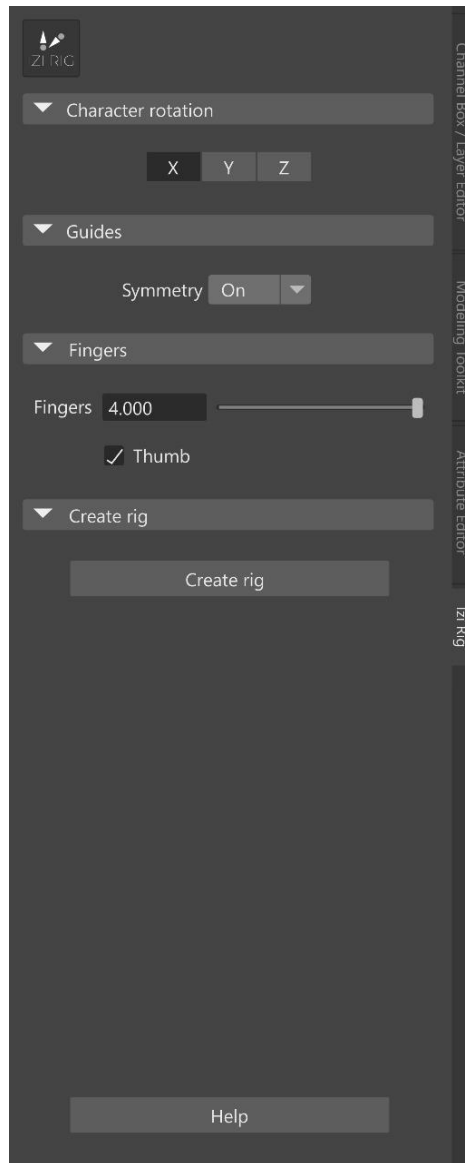


Ejemplo de el toolkit aplicado sobre un wireframe

Anexo 6







High